

063-0605-00L : Computational Structural Design 1

Computational Graphic Statics

Dr. Juney Lee & Dr. Lluís Enrique

Autumn Semester 2021

ETH zürich

DARCH

41

BRG

Week 9

Friday, November 19th

Hour 1

09:45 – 09:50

- Recap of Exercise 4

09:50 – 10:30

- **Lecture 4 : Thrust Network Analysis**

10:30 – 10:45

- Break

Hour 2

10:45 – 11:35

- **Tutorial 4 : RhinoVAULT 2 (RV2) – Part 1**

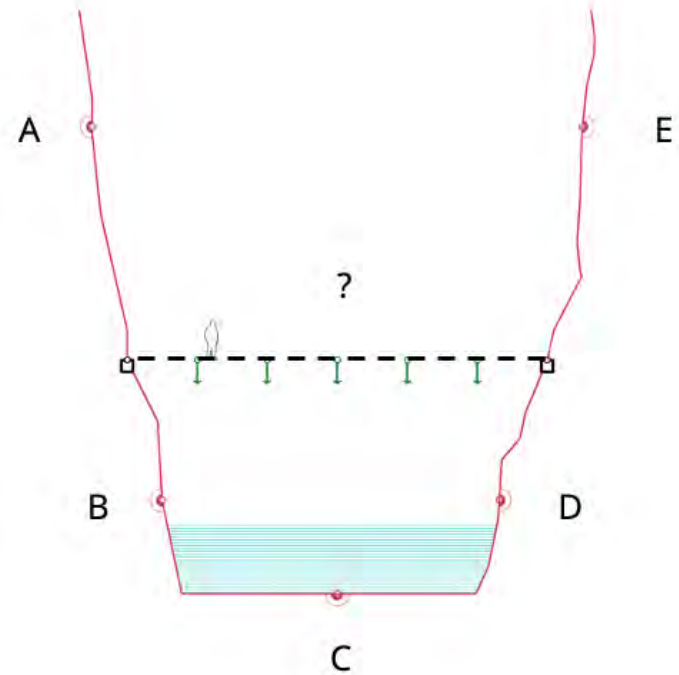
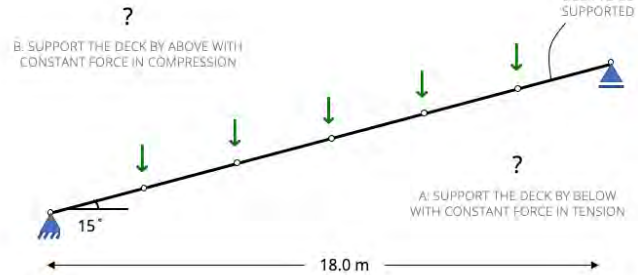
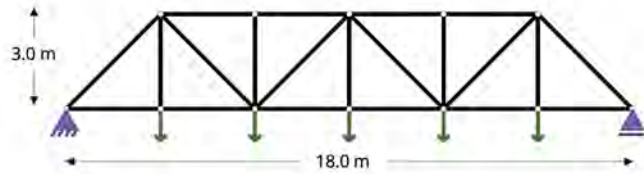
11:30 – 11:45

- Break

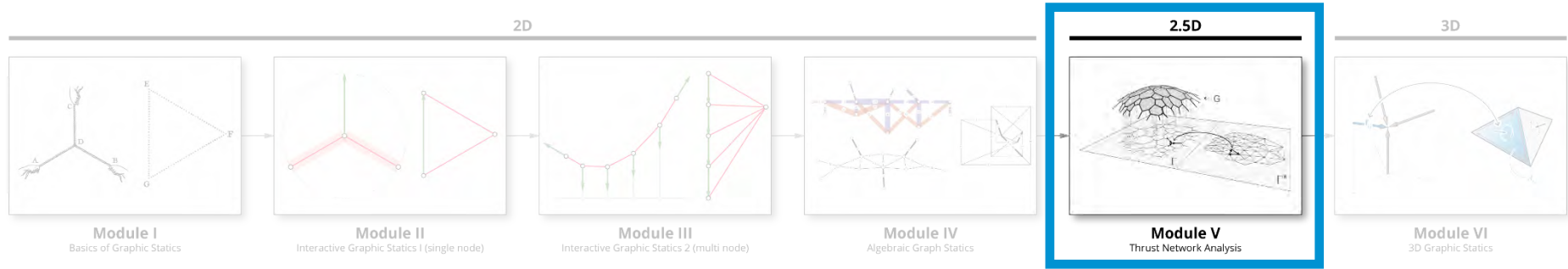
Hour 3

11:45 – 12:30

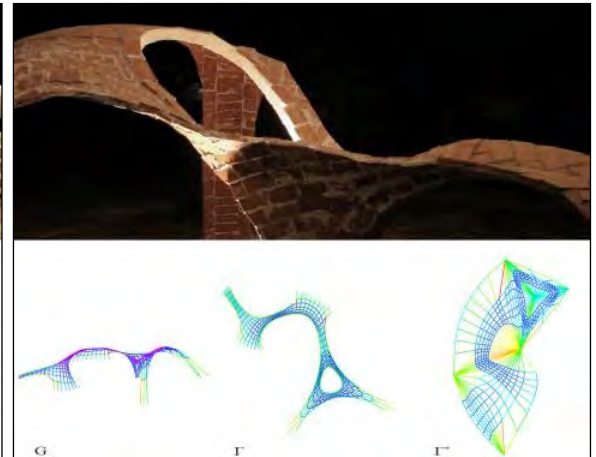
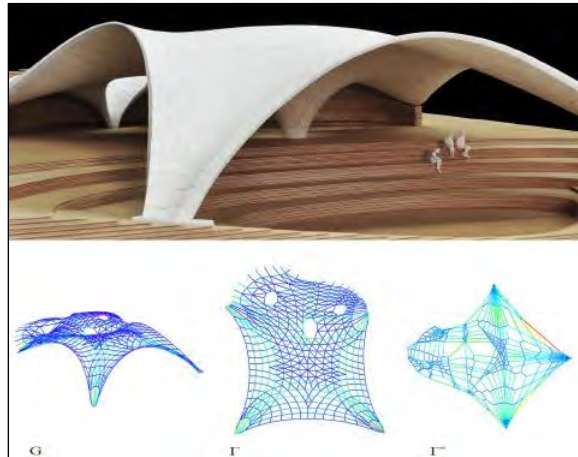
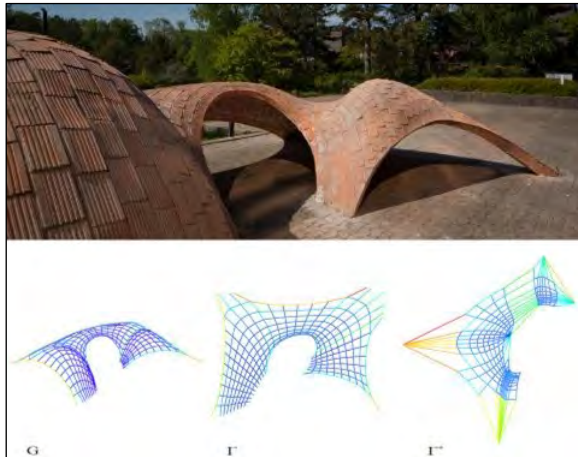
- **Tutorial 4 : RhinoVAULT 2 (RV2) – Part 2**

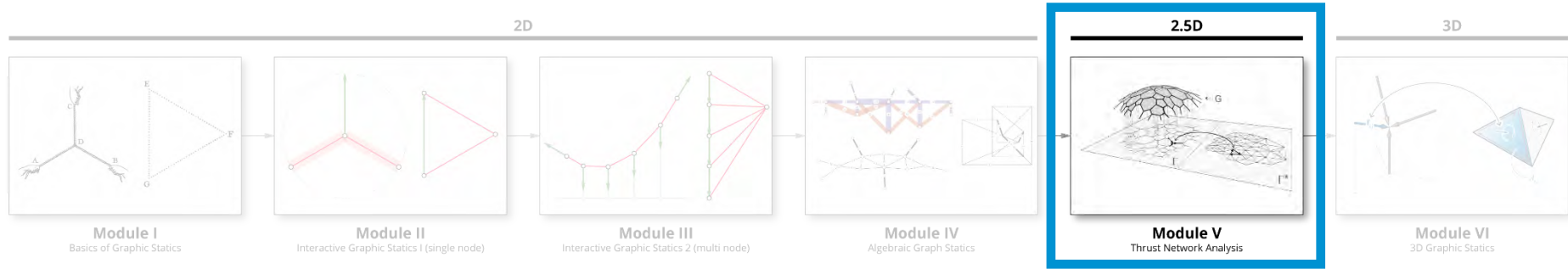


Structure Type(s)		Topic	Week(s)	Day	Topic	Instructor(s)		
1. planar structures	cables & arches	I	Week 1 (9/24)	1	Lecture 1	Introduction to graphic statics	Dr. Juney Lee	
				2	Tutorial 1	Basics of graphic statics on "blackboard"	Dr. Lluís Enríque	
				3	Work session 1	Exercise 1: Reciprocal diagrams in rhino + GH basics	Lotte Aldinger Instructors	
	2D GS (procedural)	II	Week 2 (10/1)	1	Lecture 2	Quick recap of Exercise 1	Lotte Aldinger	
				2	Tutorial 2	Algorithmic design & thinking	Dr. Juney Lee	
				3		Single node bridge in GH (form to force)	Lotte Aldinger	
			Week 3 (10/8)	1	Work session 2	Exercise 2: Single node bridge with GH	Lotte Aldinger + Instructors	
				2				
				3				
	2D GS (procedural)	III	Week 4 (10/15)	1	Lecture 3	Quick recap of Exercise 2	Lotte Aldinger	
				2	Tutorial 3	Computational graphic statics	Dr. Juney Lee	
				3		Multi-node bridge in Grasshopper	Dr. Lluís Enríque	
			Week 5 (10/22)	1	Work session 3	Exercise 3: Multi-node bridge with GH	Dr. Lluís Enríque + Instructors	
				2				
				3				
	Week 6					seminar week		
	trusses	2D GS (AGS)	IV	Week 7 (11/5)	1	Lecture 4	Quick recap of Exercise 3	Dr. Lluís Enríque
					2	Tutorial 4	Algebraic graph statics (AGS)	Dr. Juney Lee
					3		compas_agr + IGS (Interactive Graphic Statics) plugin	Ricardo Avelino
			Week 8 (11/12)	1	Work session 4	Exercise 4: Truss problems with IGS	Ricardo Avelino + Instructors	
				2				
				3				
2. surface structures	shells	2.5 GS (TNA)	V	Week 9 (11/19)	1	Lecture 5	Quick recap of Exercise 4	Ricardo Avelino
					2	Tutorial 5	Thrust Network Analysis (TNA)	Dr. Juney Lee
					3		compas_tna + RV2 (RhinoVAULT 2) plugin	Dr. Juney Lee
				Week 10 (11/26)	1			
					2			
					3			
3. spatial structures	polyhedral structures	3D GS (polyhedral)	VI	Week 11 (12/3)	1	Lecture 6	Quick recap of Exercise 5	Dr. Juney Lee
					2	Tutorial 6	3D graphic statics (3GS)	Dr. Juney Lee
					3	Work session 6	compas_3gs + 3GS (3D graphic statics) plugin	Dr. Juney Lee
				Week 12 (12/10)	1	Lecture 7	Exercise 6: Spatial structures with 3GS	
					2	Q/A	Quick recap of Exercise 6	Dr. Juney Lee
					3		Pt I. Outlook on computational graphic statics	Dr. Juney Lee
					1	Lecture 7	Pt II. Guest Lecture	TBD
					2		Discussions, feedback, evaluations, etc.	
					3			



Module V · Thrust Network Analysis & RhinoVAULT





Module V · Thrust Network Analysis & RhinoVAULT



063-0605-00L : Computational Structural Design 1
Computational Graphic Statics

Lecture 5

Thrust Network Analysis

Friday, November 19th, 2021

Dr. Juney Lee

ETH zürich

DARCH

ETH ZÜRICH
FACULTY OF ARCHITECTURE
DESIGN RESEARCH CENTER

BRG

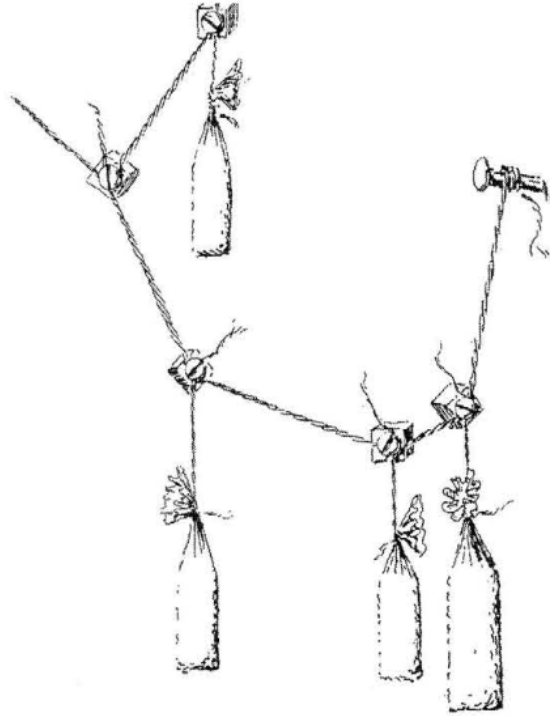
Form finding

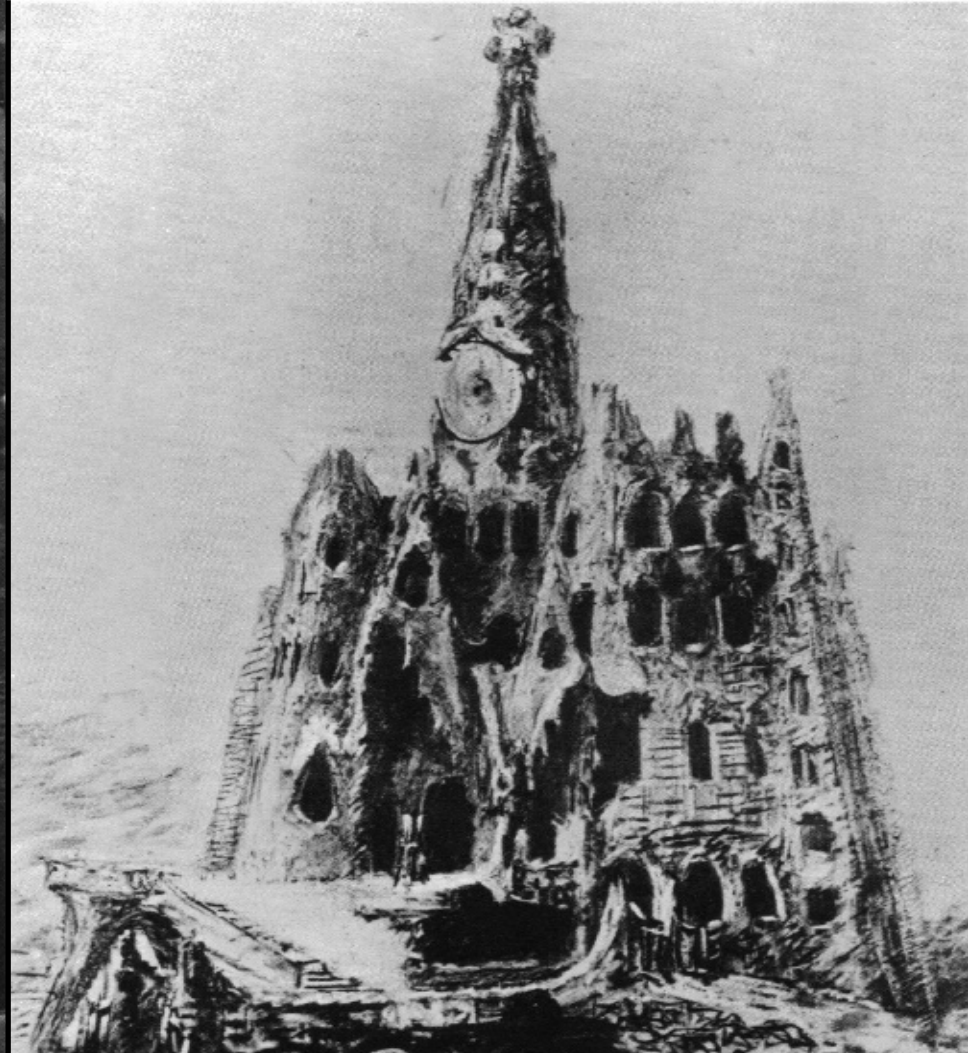
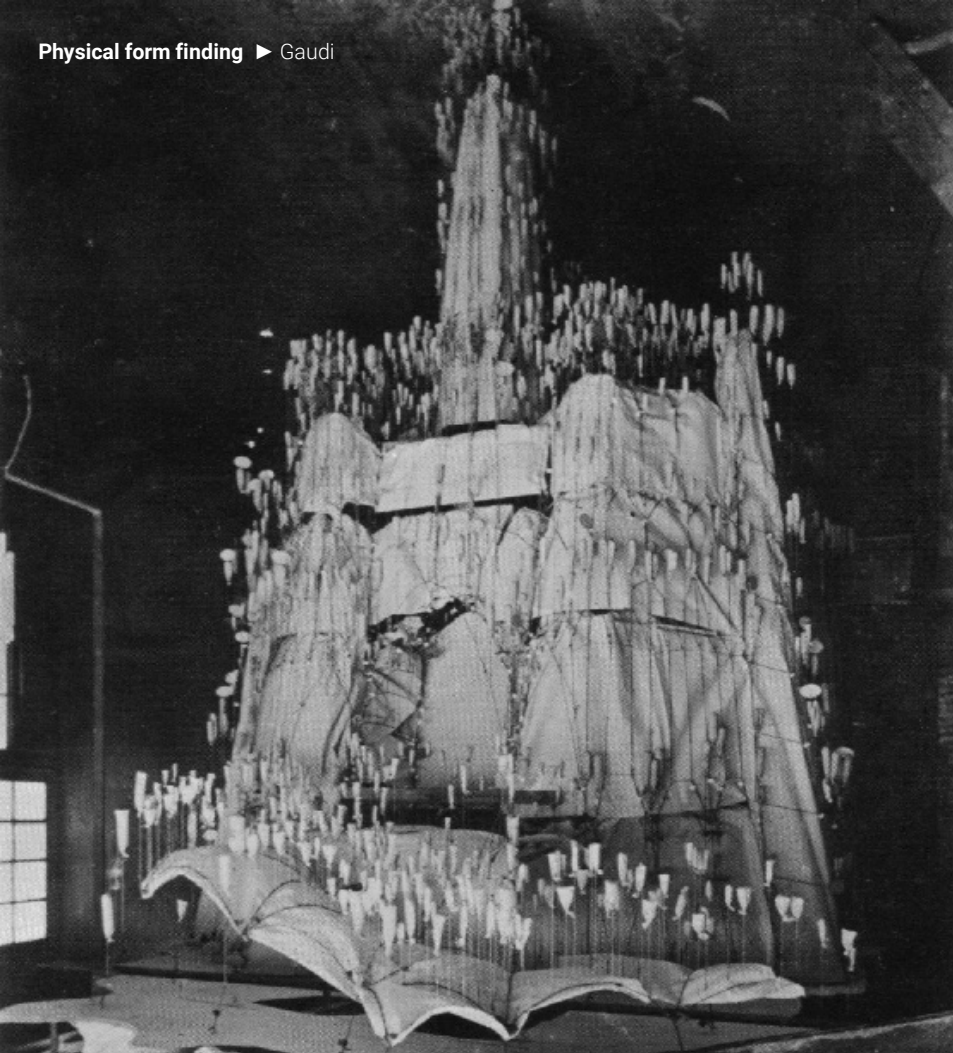


“As hangs the flexible line, so but inverted will stand the rigid arch.”

Hooke (1676)













Physical Form Finding Models. | Isler Archive



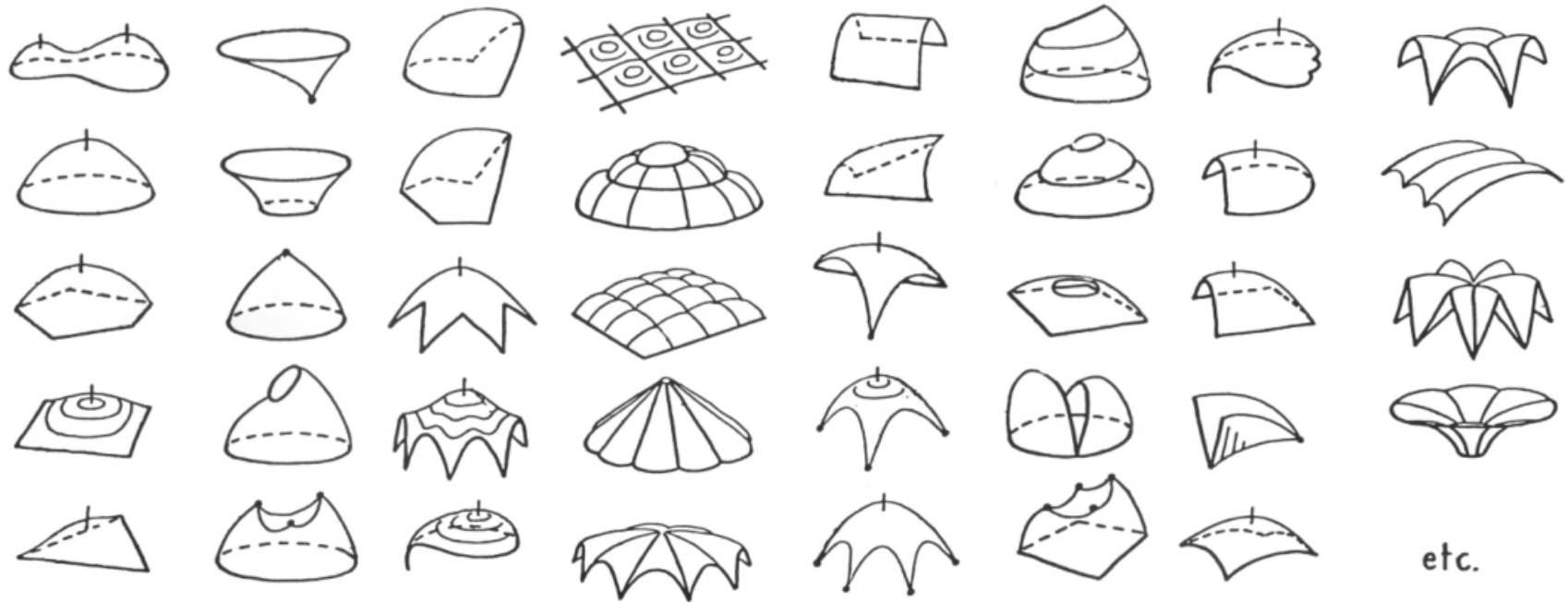
Tankstelle Deitingen Süd, Schweiz, 1968, Ing.: Heinz Isler



Tankstelle Deitingen Süd, Schweiz, 1968, Ing.: Heinz Isler



Tankstelle Deitingen Süd, Schweiz, 1968, Ing.: Heinz Isler





Soap bubble experiments | Frei Otto



German Pavilion at Expo 1967 | Frei Otto

The Institute for Lightweight Structures (IL)
University of Stuttgart

Director; Frei Otto





Montreal Pavilion measuring model, 1966



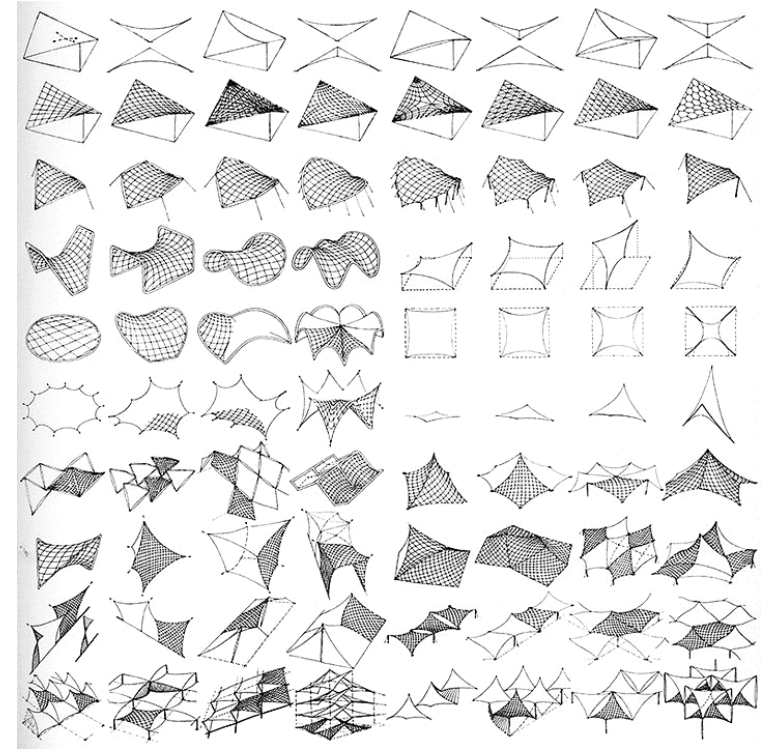
Munich Stadium measuring model, ca. 1970

Computational **form finding**

Form finding

“... forward process in which parameters are explicitly/directly controlled to find an 'optimal' geometry of a structure which is in static equilibrium with a design loading...”

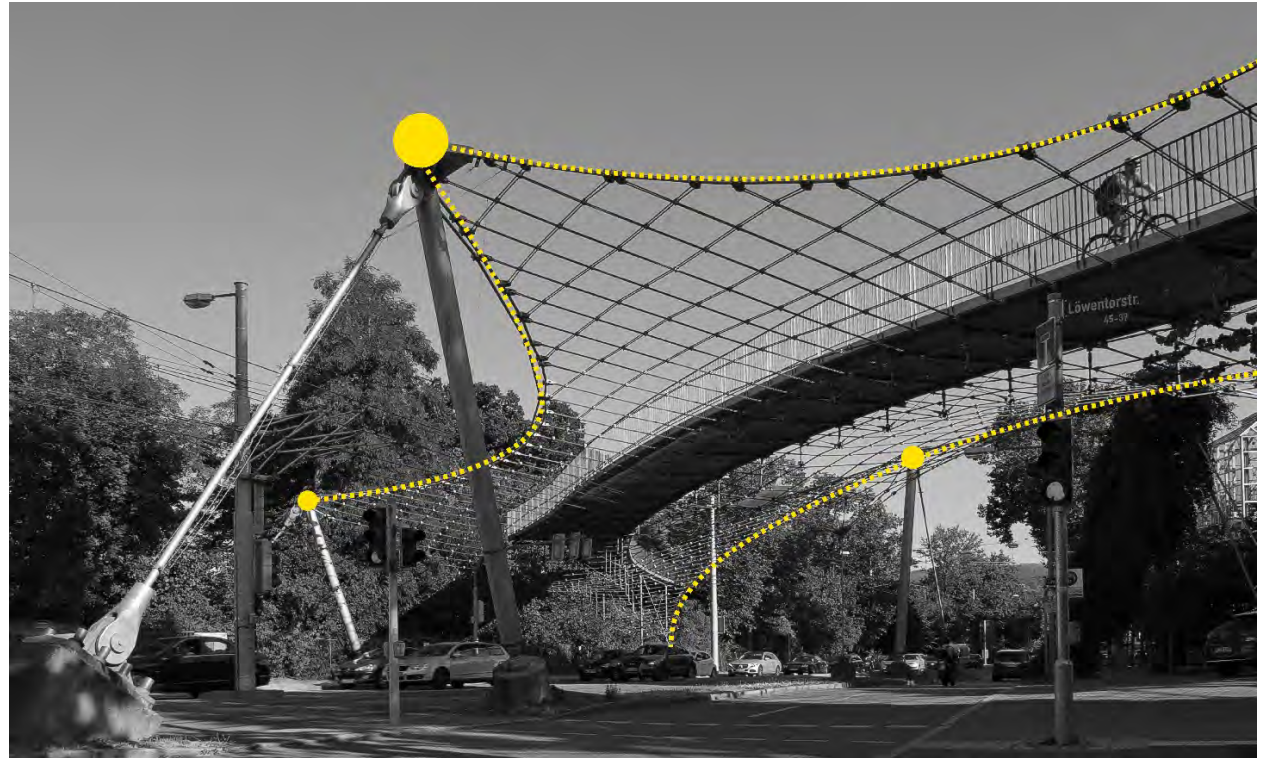
Adriaenssens et al. (2014)





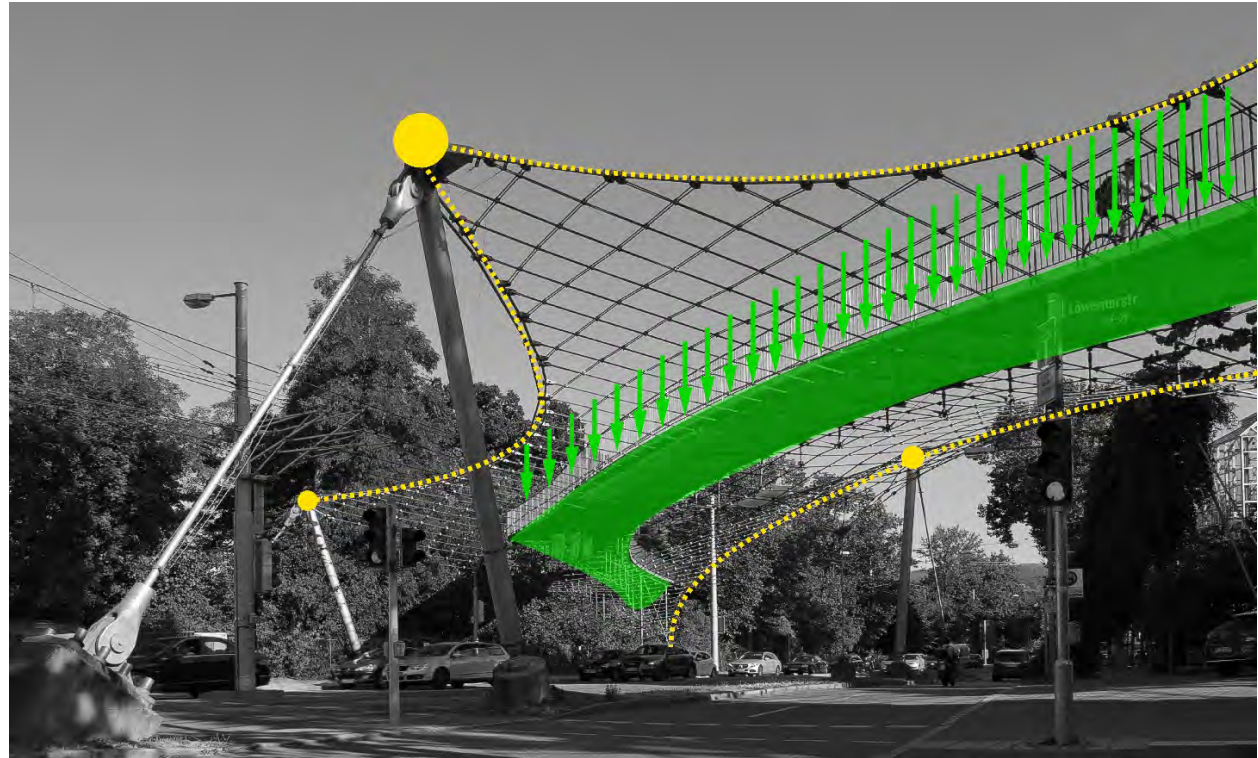
Schlaich Bergermann Partner, Lodzer Steg Stuttgart, 1992

1. Defined boundary conditions

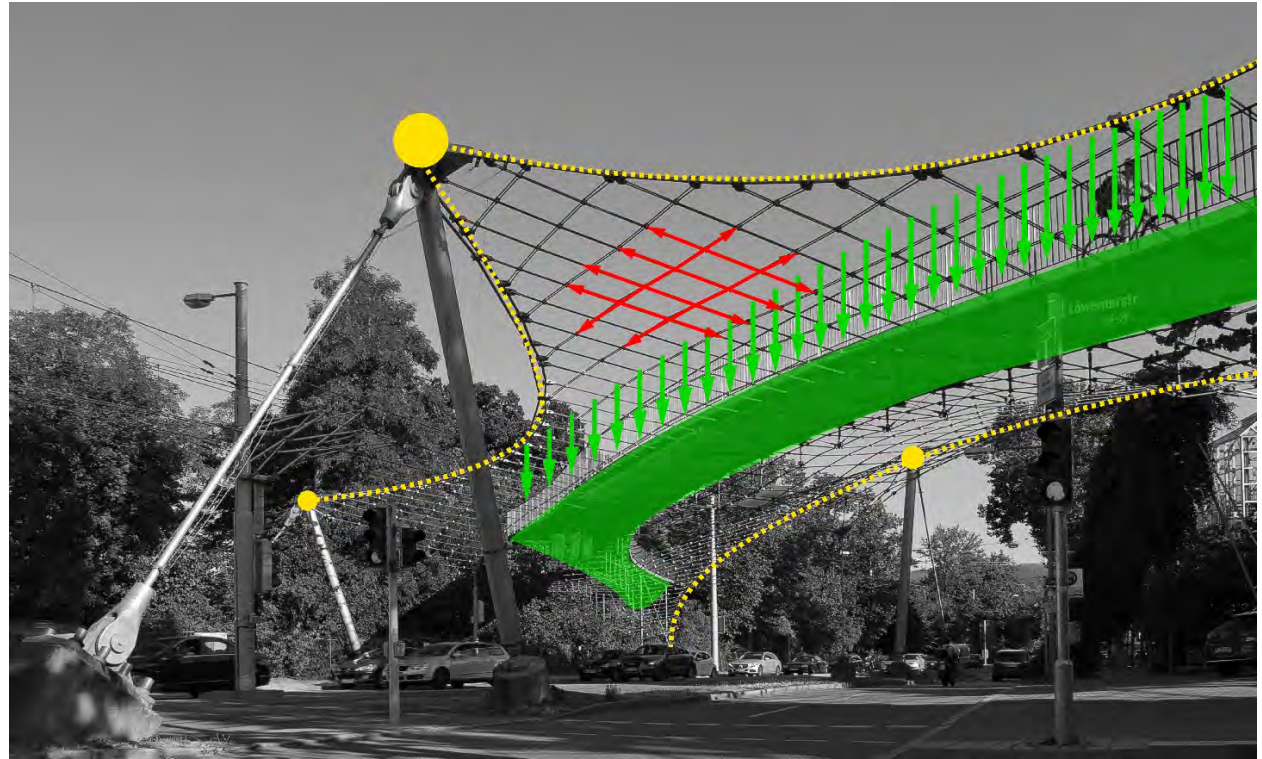


1. Defined boundary conditions

2. Defined loading case



1. Defined boundary conditions
2. Defined loading case
3. Defined state of self stress



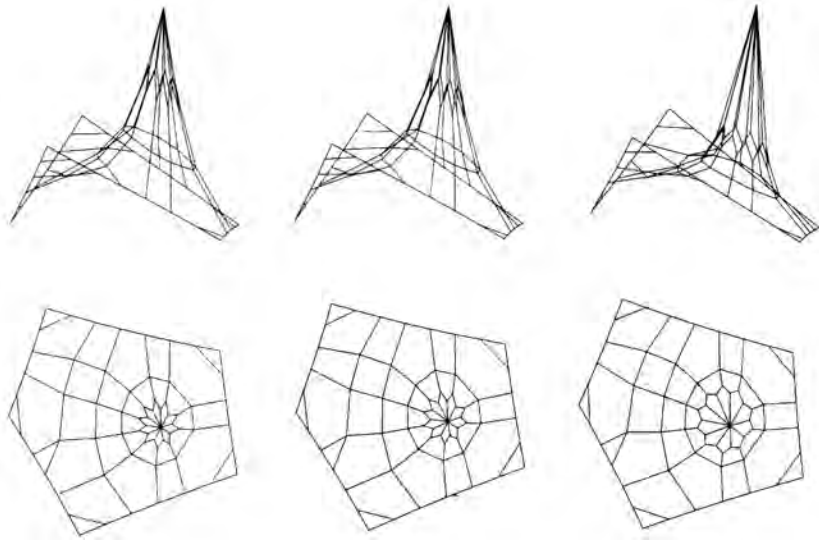


Fig. 7. Global changes in the force densities and lengthening of the radial branches.

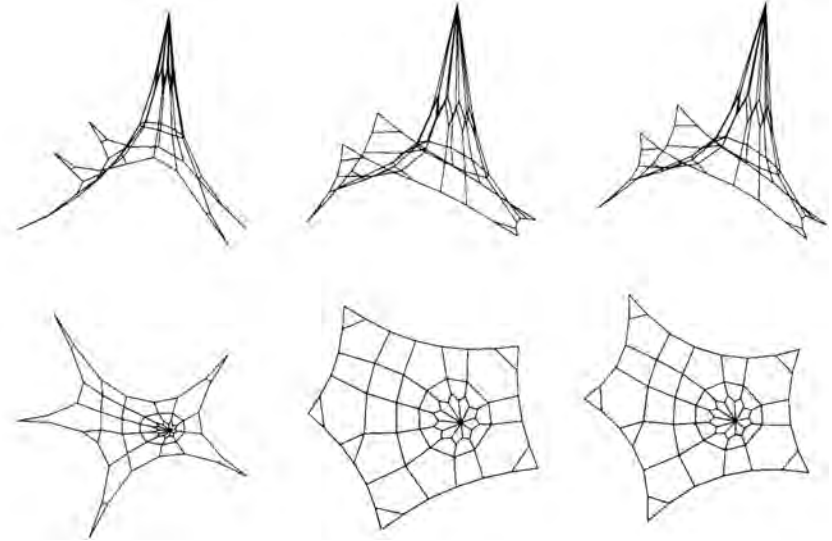
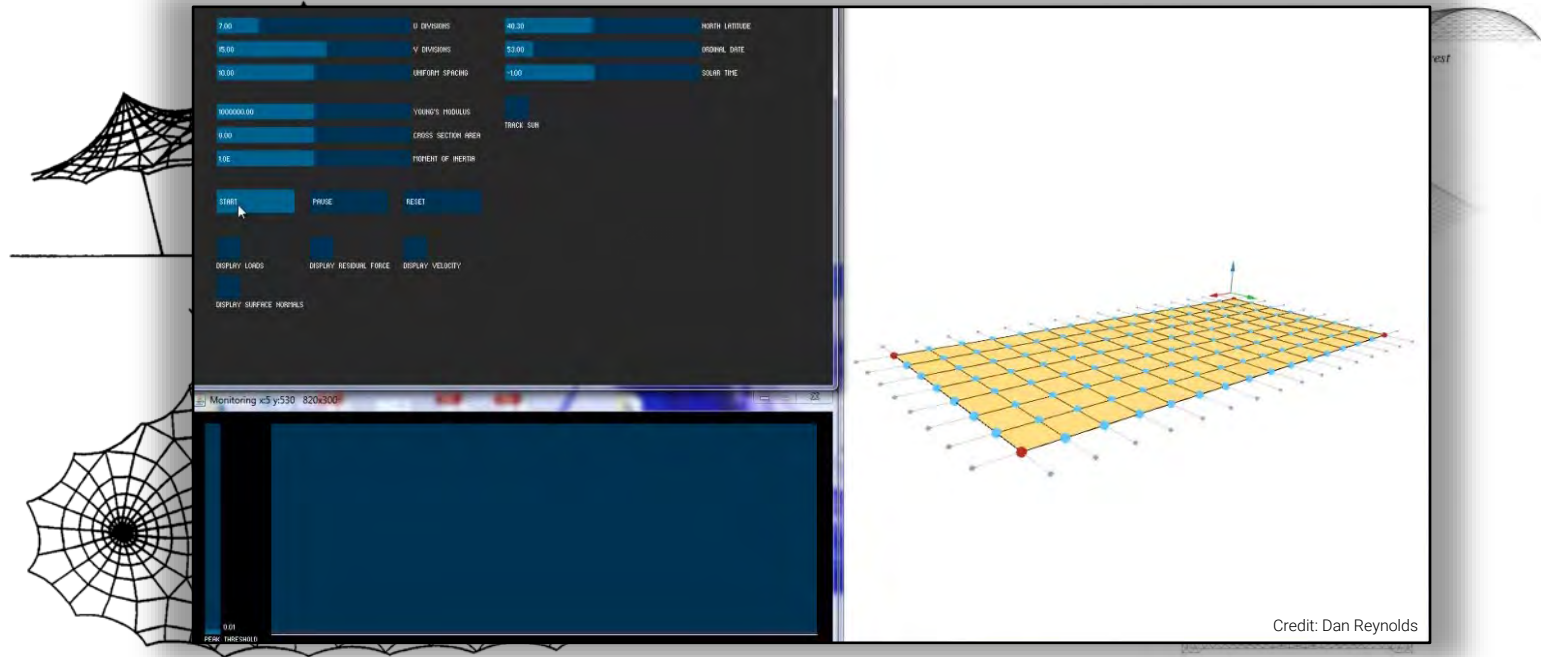


Fig. 8. Global changes in the force densities of the main cables.



Credit: Dan Reynolds

Figure 13. Outwards deflections due to loading

Form finding with graphic statics

THRUST NETWORK ANALYSIS: A NEW METHODOLOGY FOR THREE-DIMENSIONAL EQUILIBRIUM

PHILIPPE BLOCK¹ AND JOHN OCHSENDORF²

¹Research Assistant, ph_block@mit.edu, ²Associate Professor, jand@mit.edu, Building Technology Program, Massachusetts Institute of Technology, Room 5-419, 77 Massachusetts Avenue, Cambridge, MA 02139, USA

Editor's Note: The first author of this paper is one of the four winners of the 2007 Hargai Prize, awarded for outstanding papers that are submitted for presentation and publication at the annual IASS Symposium by younger members of the Association (under 30 years old). It is re-published here with permission of the proceedings of the IASS 2007 Symposium: Structural Architecture: Toward the Future Looking to the Past, held in December 2007 in Venice, Italy.

ABSTRACT

This paper presents a new methodology for generating compression-only vaulted surfaces and networks. The method finds possible funicular solutions under gravitational loading within a defined envelope. Using projective geometry, duality theory and linear optimization, it provides a graphical and intuitive method, adapting the same advantages of techniques such as graphic statics, but offering a viable extension to fully three-dimensional problems. The proposed method is applicable for the analysis of vaulted historical structures, specifically in unreinforced masonry, as well as the design of new vaulted structures. This paper introduces the method and shows examples of applications in both fields.

Keywords: compression-only structures, unreinforced masonry vaults, funicular analysis, Thrust Network Analysis, reciprocal diagrams, form-finding, lower-bound analysis, structural optimization

1. INTRODUCTION

Medieval vault builders created complex forms carefully balanced in compression. The structural properties of these sophisticated forms are still poorly understood because of a lack of appropriate analysis methods, i.e. methods relating stability and form. Understanding the mechanics of these vaulted structures leads to new insights for both analysis and design.

Thrust Line Analysis is a powerful graphical method for calculating the range of lower-bound equilibrium solutions of compression-only systems, such as unreinforced masonry structures (Fig. 1). It visualizes the stability of these structures and suggests possible collapse mechanisms [1]. Unfortunately, thrust line analysis is primarily suitable for 2-D cases and this limitation has prevented it from being used for the assessment of complex 3-D structures. While numerical methods

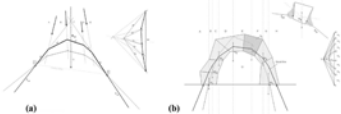


Figure 1. (a) Two possible compression-only equilibrium shapes for a random set of loads, and (b) an interactive thrust-line application developed in [1]: the user can adapt the geometry by dragging control points and the structural feedback, in the form of a thrust-line, is updated in real-time. The magnitudes of the forces in the system are visualized in the accompanying funicular polygon (right).

based on elastic tolerances give one possible answer, they no longer suggest better form as was adhered to (the more holistic graphical methods).

There is a real need for tools to better understand and visualize the stability of compression-only structures, such as historic unreinforced masonry structures, as well as design tools that suggest better form. Both problems are related to finding vault force structures in equilibrium acting only in compression or tension. Currently, graphic statics provides a holistic analysis and design tool for two-dimensional structures. With today's availability of powerful virtual 3-D and parametric environments, the following question arises: can a fully three-dimensional version of thrust-line analysis provide the same freedom to explore the infinite equilibrium solutions for a certain loading condition?

2. METHODOLOGY

The Thrust-Network Analysis method presented in this paper is inspired by O'Dwyer's work on funicular analysis of vaulted masonry structures [2]. It is extended by adding the concept of duality between geometry and the in-plane internal forces of networks [3].

2.1. Reciprocal figures

The duality between the geometry of a network and its internal forces is an old concept, first explained by Maxwell [4]. He called this relationship reciprocal and defined it as follows: "Two plane figures are reciprocal when they consist of an equal number of lines, so that corresponding lines in the



Figure 2. The primal grid Γ and dual grid Γ^* are related by a reciprocal relationship. The equilibrium of a node in one of them is guaranteed by a closed polygon in the other and vice versa. The labeling uses these's notation [4].

two figures are parallel, and corresponding lines which converge to a point in one figure form a closed polygon in the other". This means that the equilibrium of a node in the first diagram is represented by a closed polygon in the second diagram and vice versa (Fig. 2). Graphic statics is based on this principle [5].

2.2. Assumptions

The proposed method produces funicular (compression-only) solutions for loading conditions where all loads are applied in the same direction, as is the case for gravitational loading. Since the solutions are compression-only this also means that the vaults can never curl back onto themselves, which would demand some elements to go into tension. The resulting three-dimensional networks can represent load paths throughout a structure. There is no constraint on the length of the branches

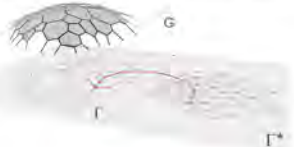


Figure 3. Relationship between compression shell $S(x)$ in plane projections (primal grid Γ) and the reciprocal diagram, dual grid Γ^* as kinematic equilibrium.



Figure 7. (a) Possible thrust values for this given vault range from 21% to 42% of its total weight. (b) 3-d web and rib action with the forces mainly spanning between the ribs as represented in the dual grid (c).

(a) calm of lower-bound analysis. Put simply, if a compression-only network can be found that fits within the boundaries of a vault, then the vault will stand in compression. Note that if we want a solution without any tension (hinges) in the section, we want the solution to lie within the middle third of the section (defined by equation 6). This is a powerful concept for understanding the stability and proximity to collapse of such structures. Additional reading on this topic can be found in Heyman [10], O'Dwyer [2], Boothby [11] and Block et al. [1].

Figure 7b shows a solution with three-dimensional web action. The distribution of the horizontal components in the network is represented in its dual grid (Fig. 7c). More detailed applications of this methodology in masonry vaults can be found in [12].

The method is particularly appropriate for historic masonry vaults because their self-weight is the dominant load. The range of possible equilibrium states, bounded by a minimum and maximum thrust, can be produced (Fig. 7a). We are interested in this range of thrust values because it provides the

most useful characterization of the structural behavior of the vault. The minimum (or passive) thrust state represents the least amount this vault will push sideways onto its neighboring elements, as a function of its self-weight and shape. The maximum (or active) state of thrust on the other hand represents the largest horizontal force this vault can provide. So, this value demonstrates how much horizontal force this vault can safely take from its neighboring elements.

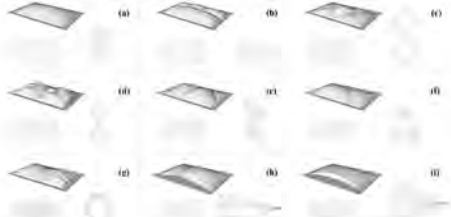


Figure 8. A series of examples, starting from a regular rectangular grid, showing the relationship between the primal and dual grid and the corresponding patterns.

Thrust Network Analysis

Exploring Three-dimensional Equilibrium

by

Philippe Block

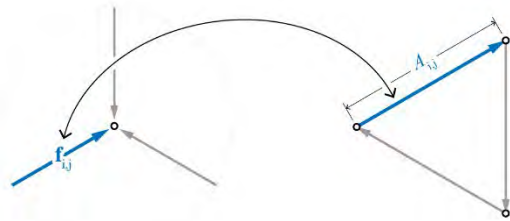
Submitted to the Department of Architecture
on May 1, 2009 in Partial Fulfillment of the Requirements
for the Degree of Doctor of Philosophy in Architecture: Building Technology

Abstract

This dissertation presents Thrust Network Analysis, a new methodology for generating compression-only, vaulted surfaces and networks. The method finds possible funicular solutions under gravitational loading within a defined envelope. Using projective geometry, duality theory and linear optimization, it provides a graphical and intuitive method, adopting the same advantages of techniques such as graphic statics, but offering a viable extension to fully three-dimensional problems. The proposed method is applicable for the analysis of vaulted historical structures, specifically in masonry, as well as the design of new vaulted structures. This dissertation introduces the method and shows examples of applications in both fields.

Thrust Network Analysis, masonry, historic structures, compression-only structures, limit analysis, equilibrium analysis, funicular design, force finding, structural optimization, Gothic vaults, reciprocal diagrams.

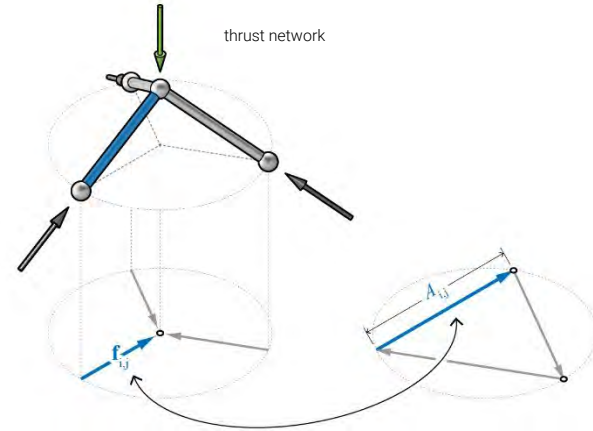
Thesis supervisor: John A. Ochsendorf
Title: Associate Professor of Building Technology



form diagram

force diagram

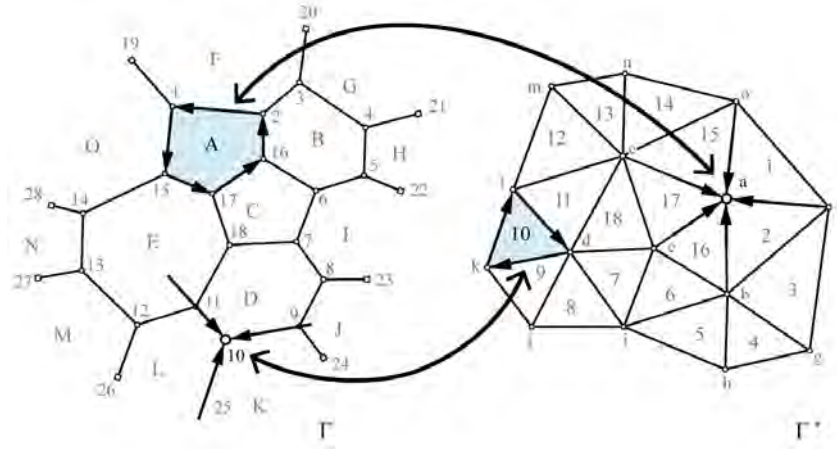
2 D



form diagram

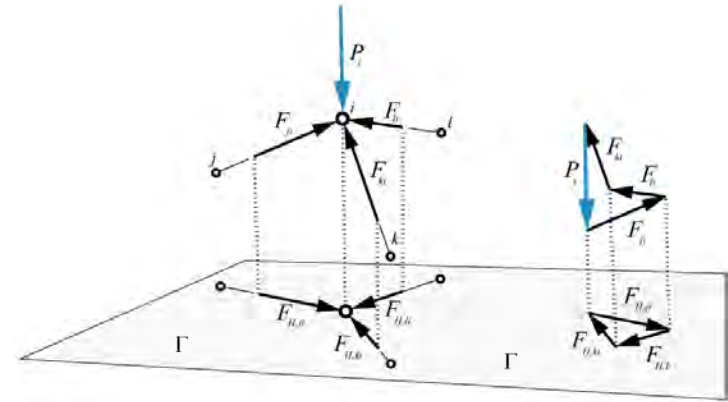
force diagram

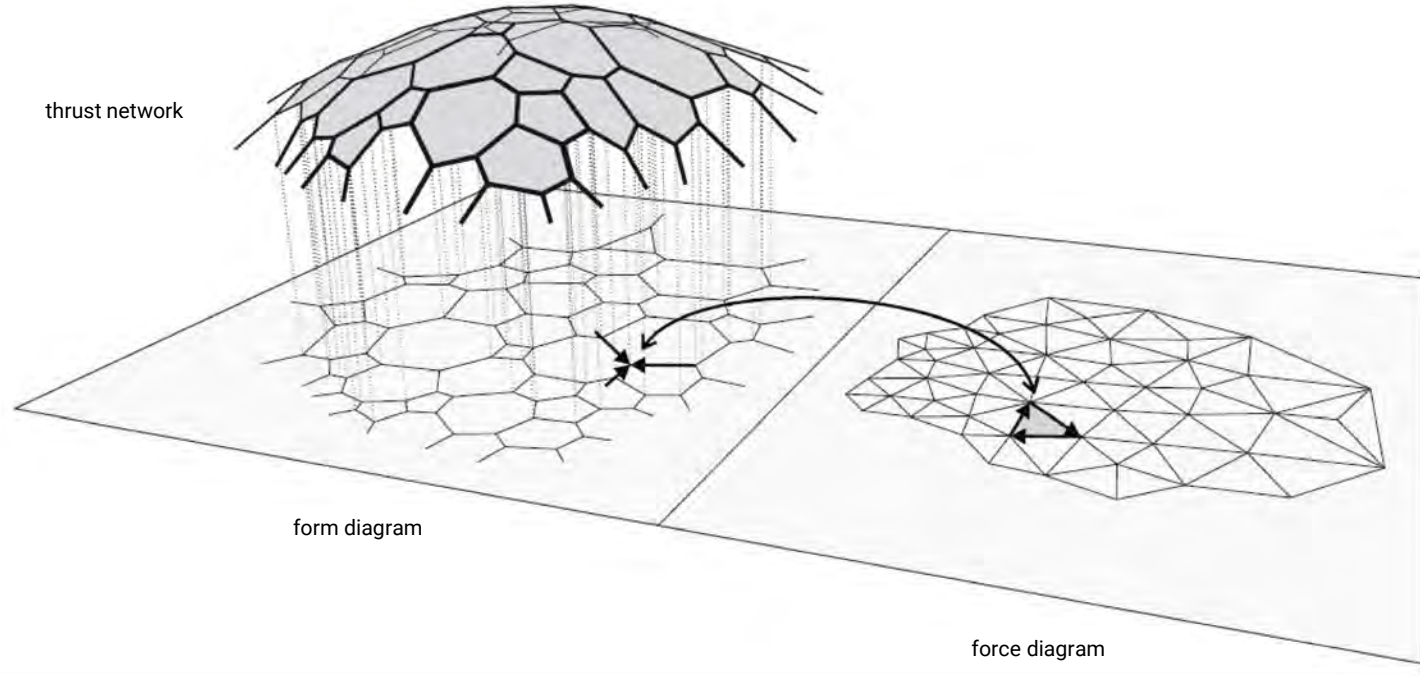
2.5 D

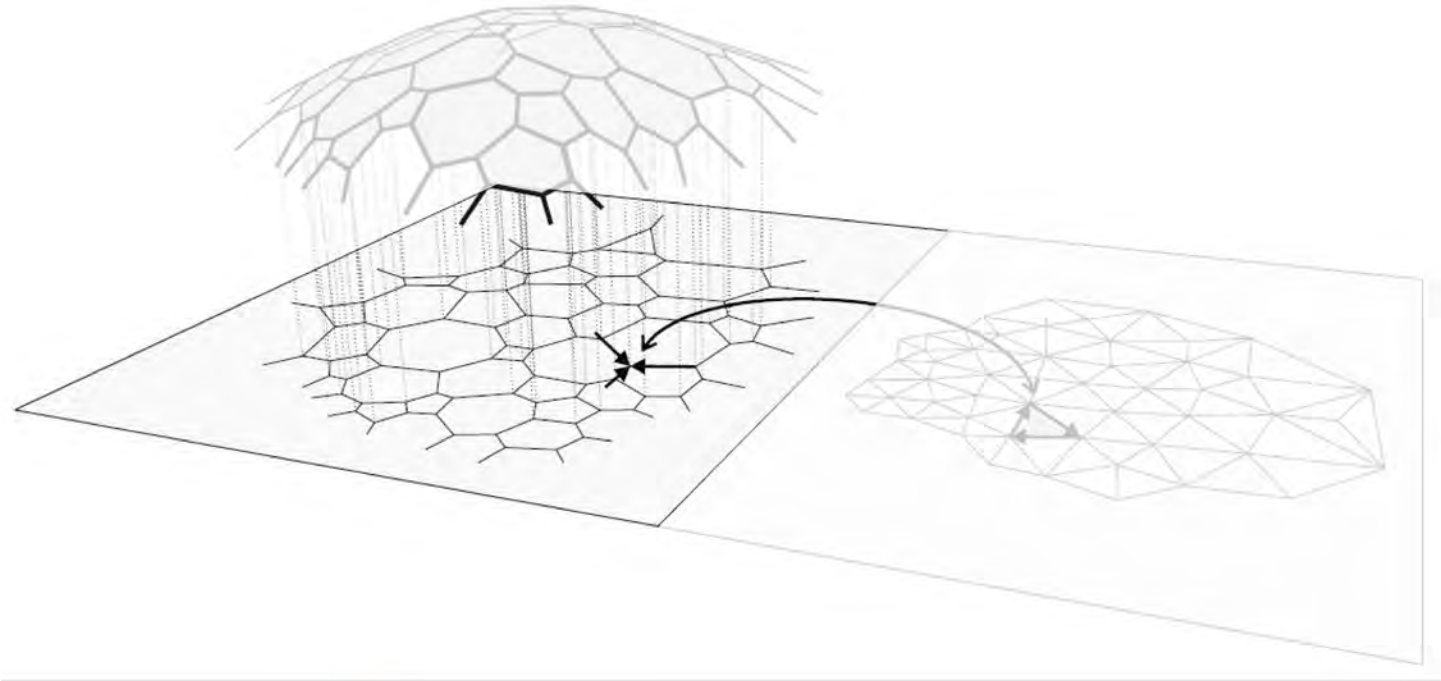


Form Diagram

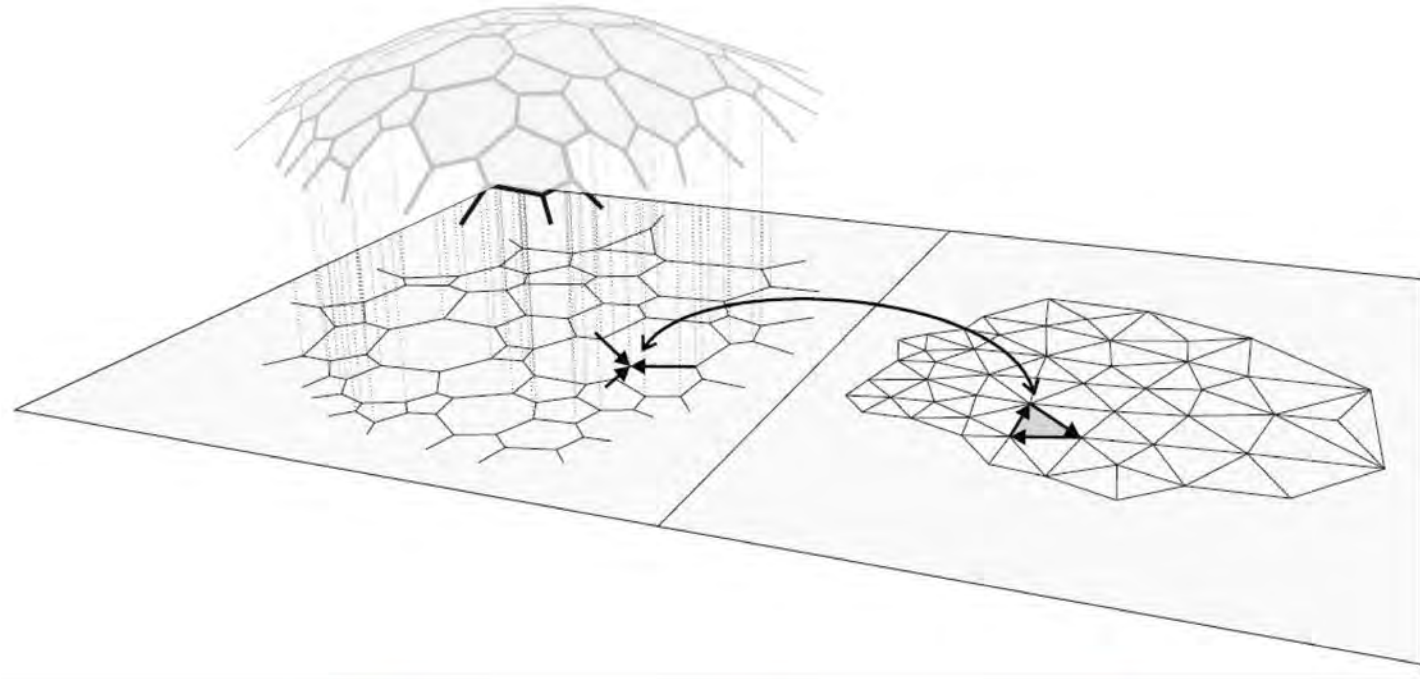
Force Diagram

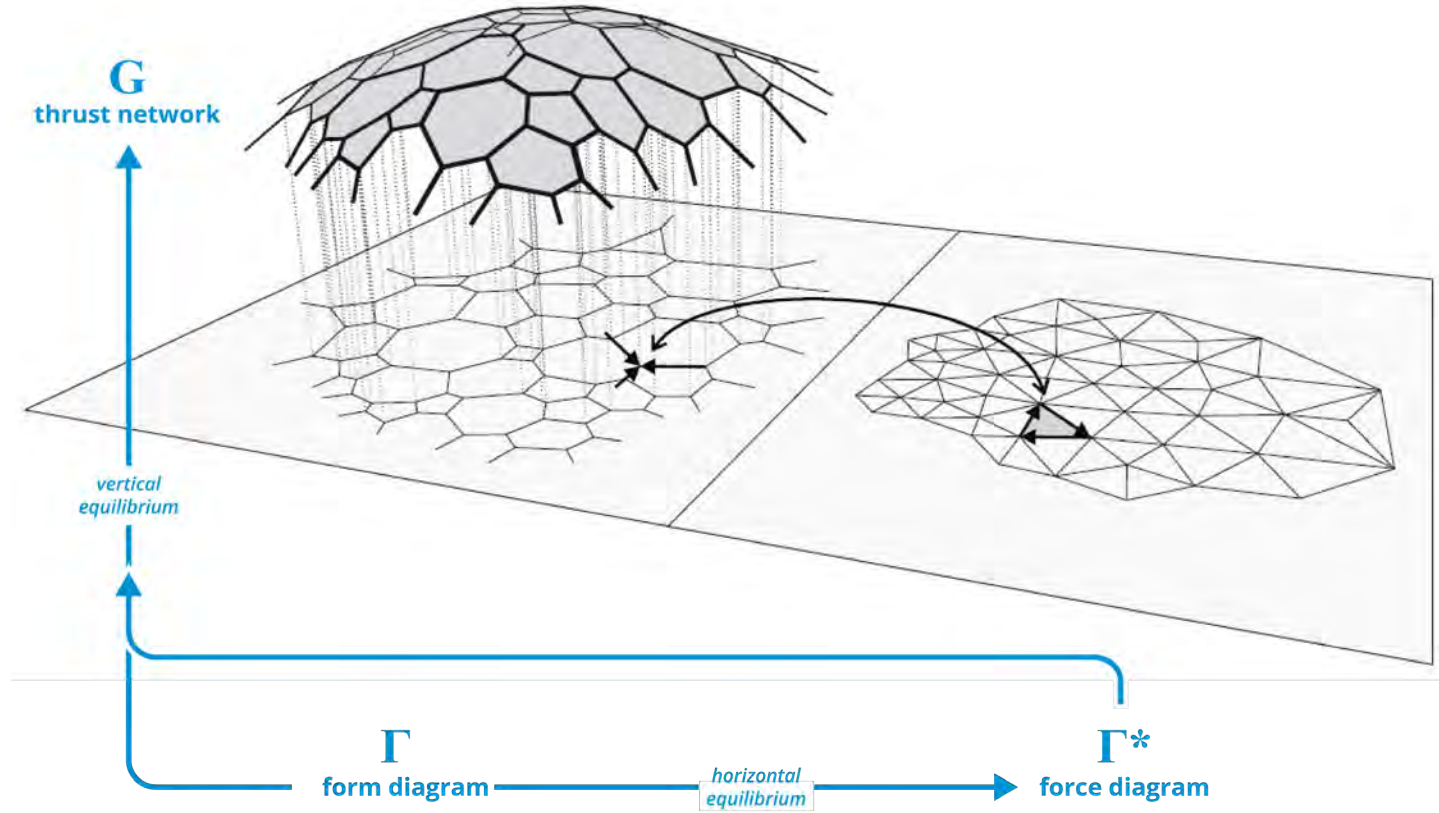




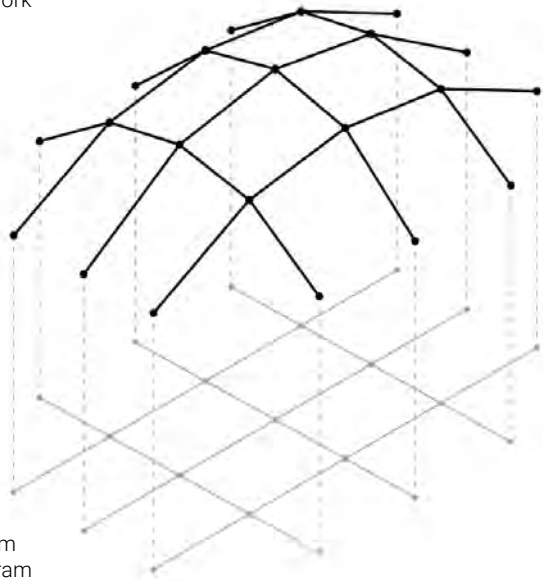


Γ
form diagram

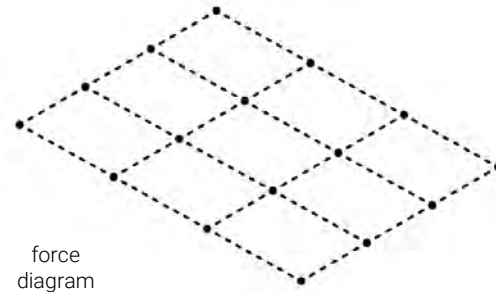




thrust
network

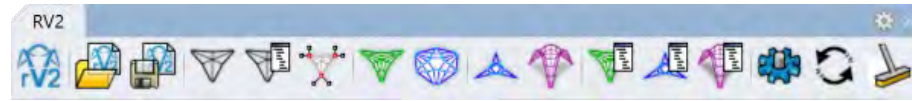


form
diagram



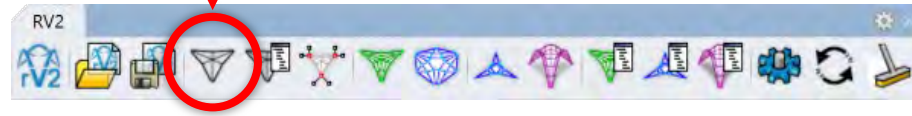
force
diagram

RV2 workflow



TNA steps

make pattern



1. Pattern

2. Boundary conditions

3. Form diagram

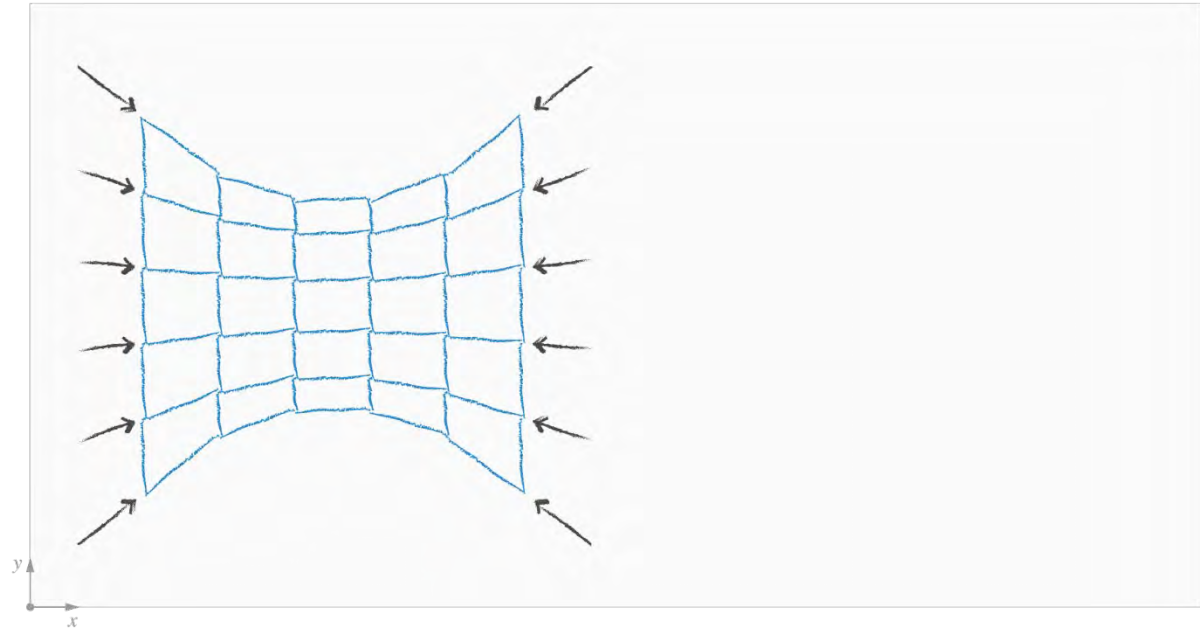
4. Dual diagram

5. Horizontal equilibrium

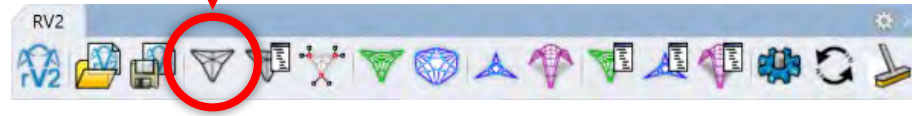
6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



make pattern



1. Pattern

2. Boundary conditions

3. Form diagram

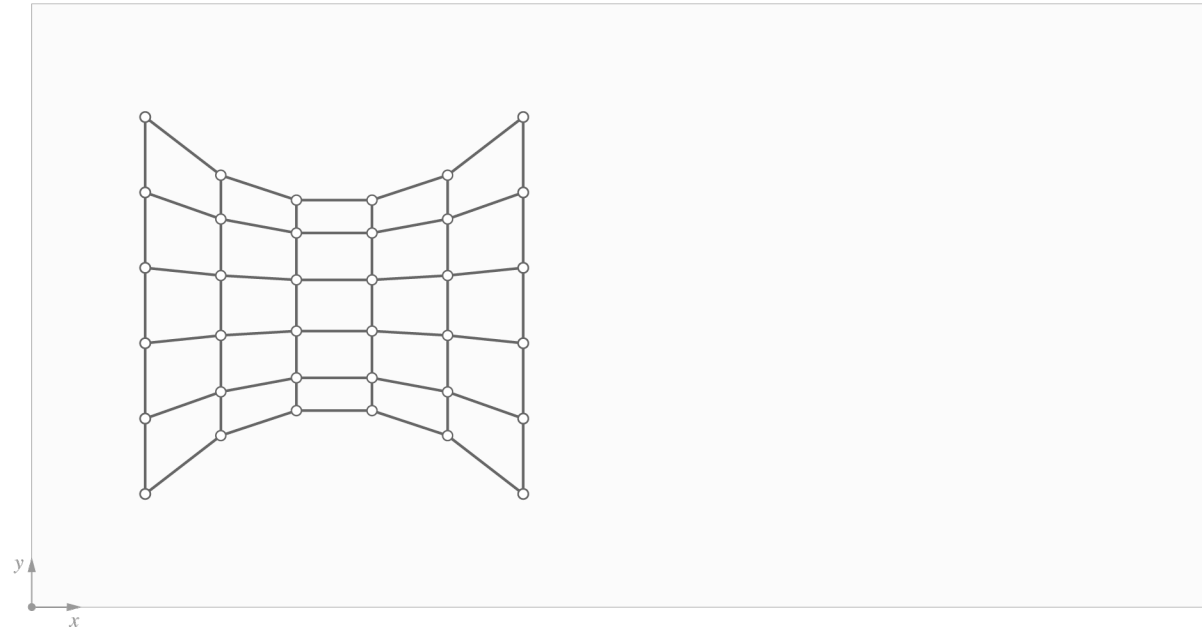
4. Dual diagram

5. Horizontal equilibrium

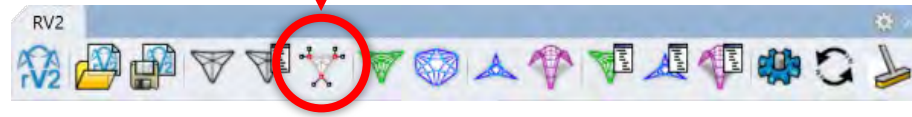
6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



define boundary conditions



1. Pattern

2. Boundary conditions

3. Form diagram

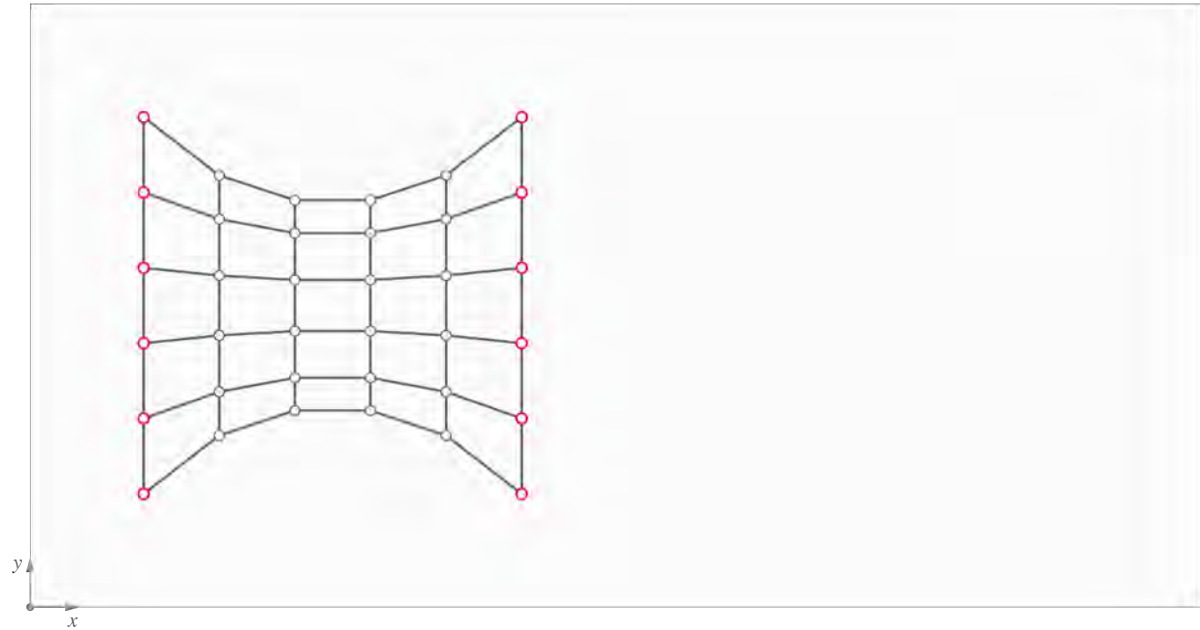
4. Dual diagram

5. Horizontal equilibrium

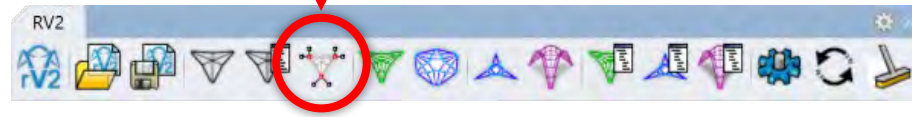
6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



define boundary conditions



1. Pattern

2. Boundary conditions

3. Form diagram

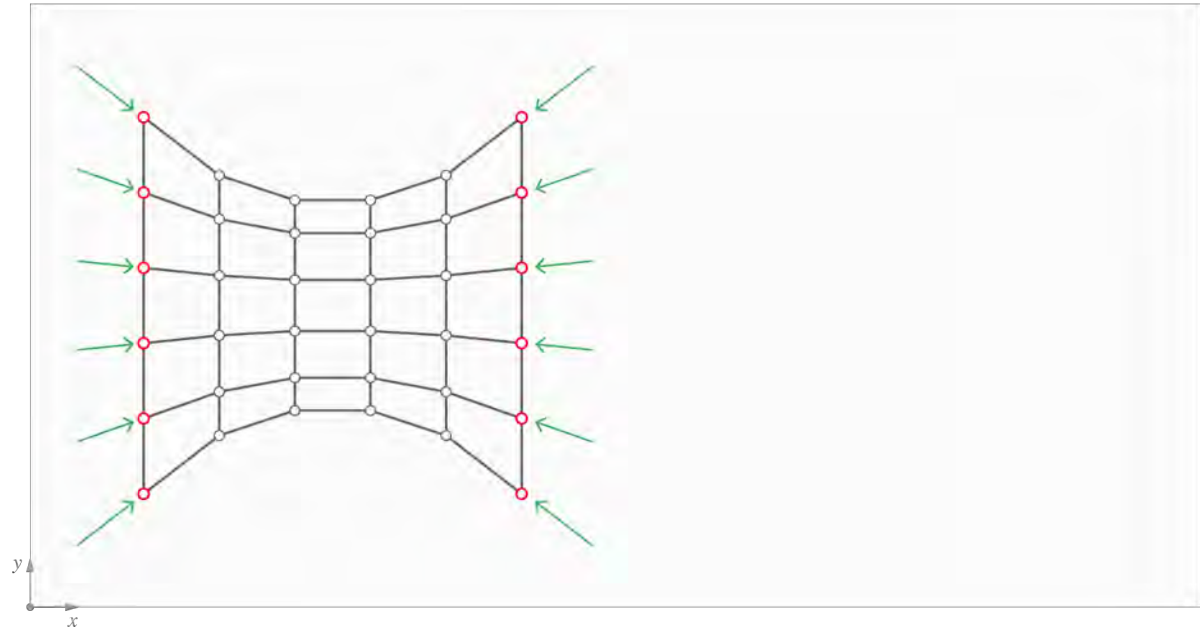
4. Dual diagram

5. Horizontal equilibrium

6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



form diagram



1. Pattern

2. Boundary conditions

3. Form diagram

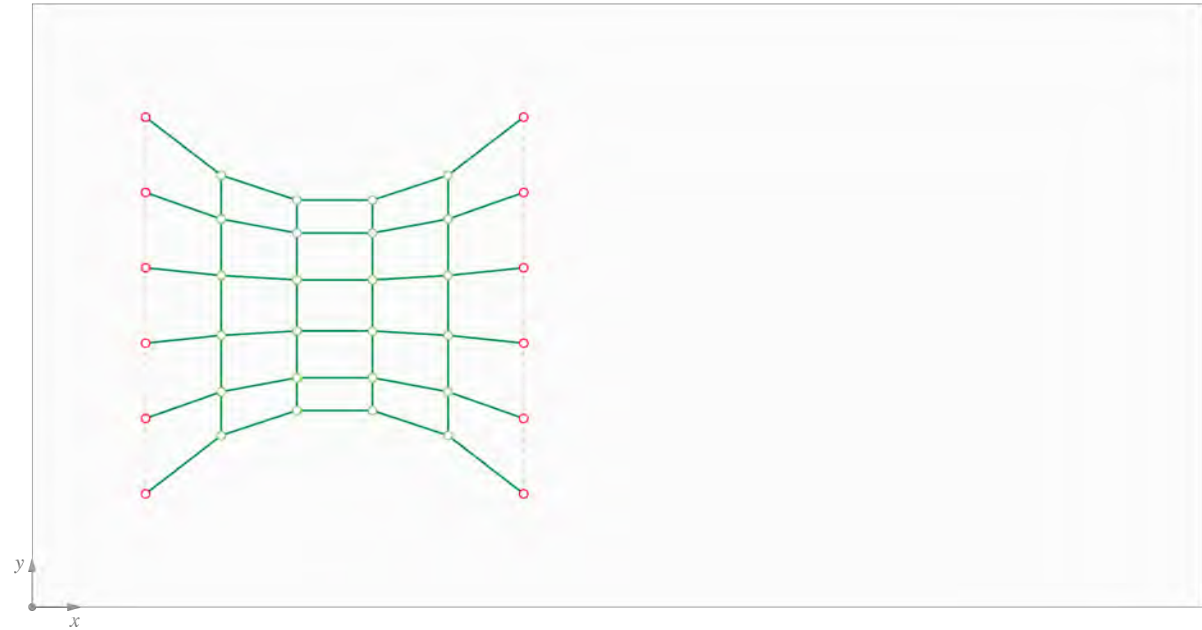
4. Dual diagram

5. Horizontal equilibrium

6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



dual diagram



1. Pattern

2. Boundary conditions

3. Form diagram

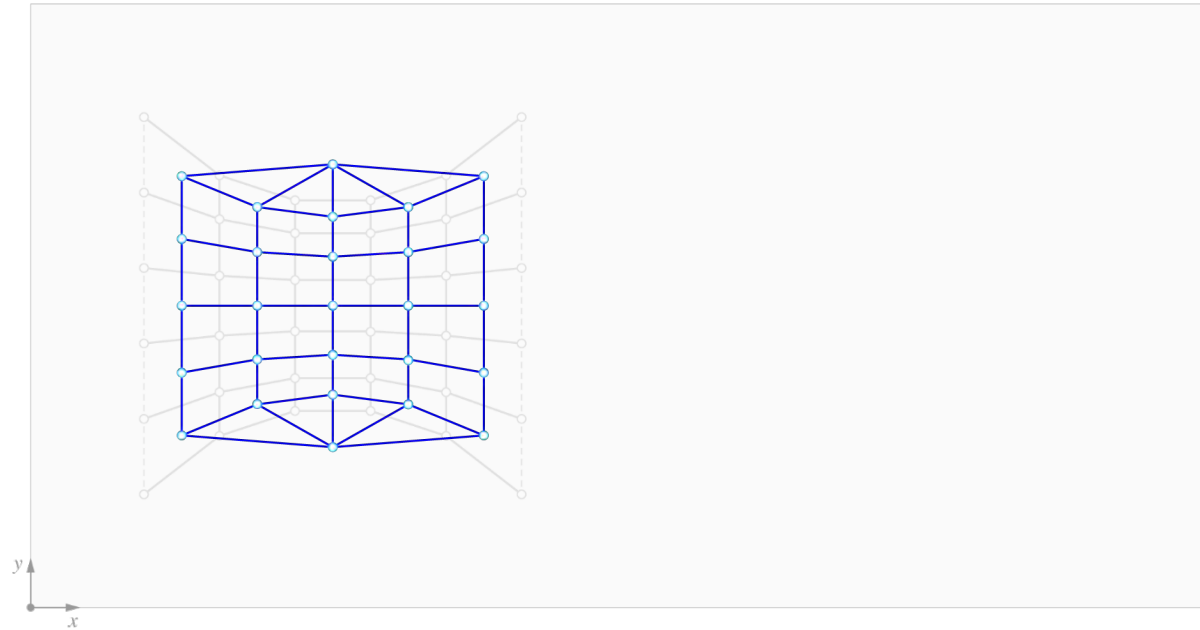
4. Dual diagram

5. Horizontal equilibrium

6. Force diagram

7. Vertical equilibrium

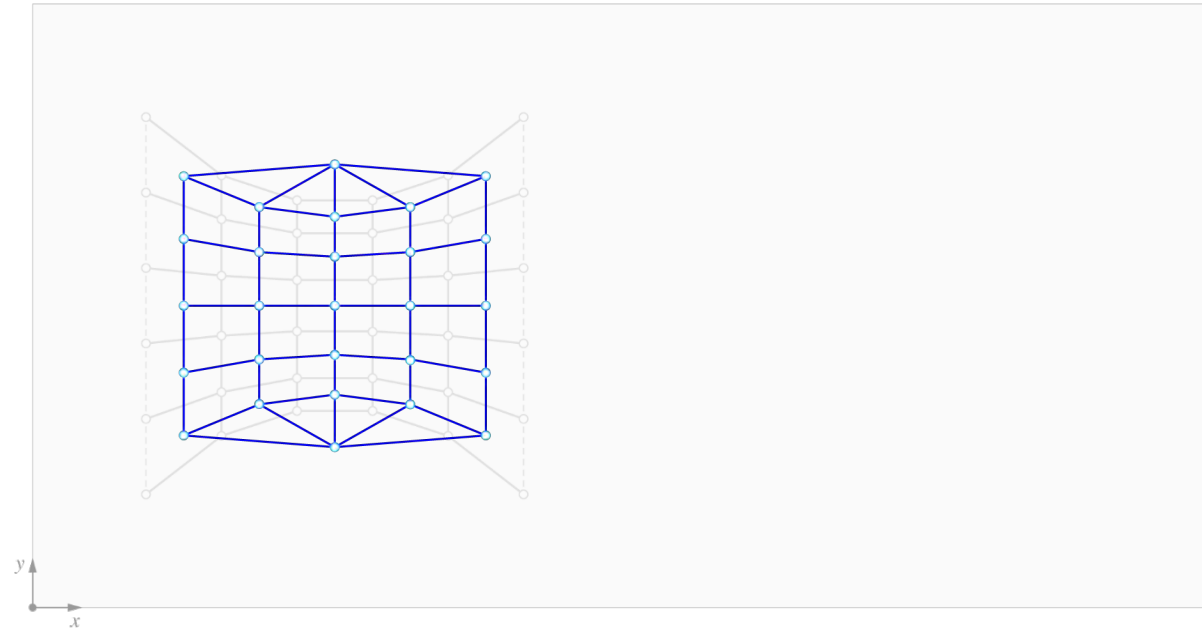
8. Thrust diagram



dual diagram



1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



dual diagram



1. Pattern

2. Boundary conditions

3. Form diagram

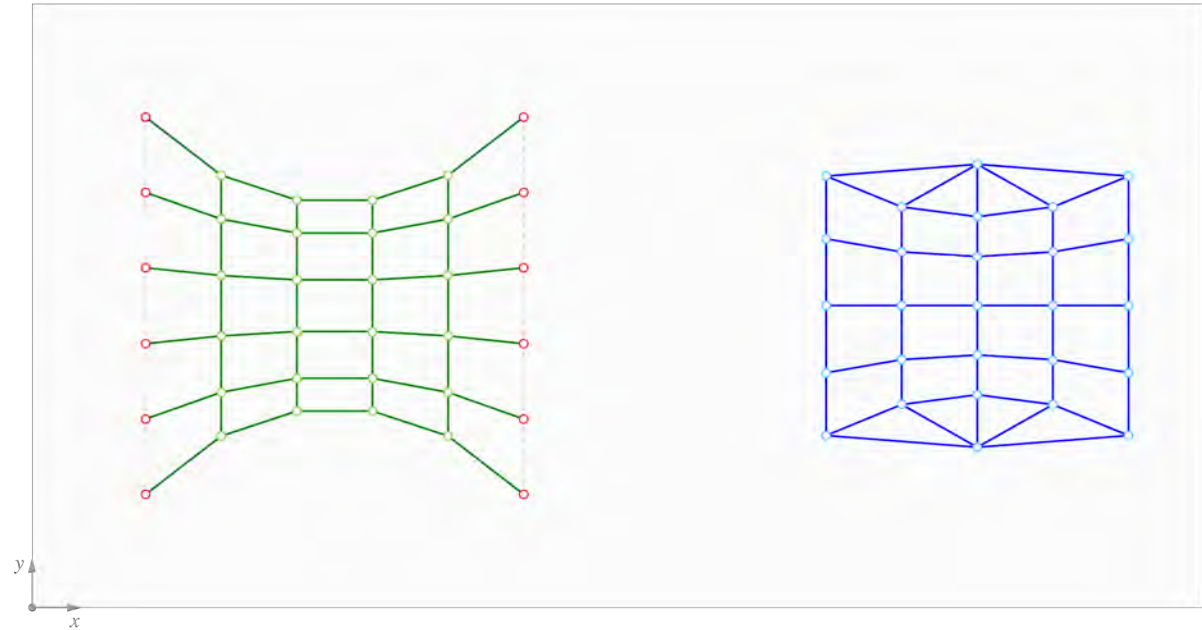
4. Dual diagram

5. Horizontal equilibrium

6. Force diagram

7. Vertical equilibrium

8. Thrust diagram



dual diagram



1. Pattern

2. Boundary conditions

3. Form diagram

4. Dual diagram

5. Horizontal equilibrium

6. Force diagram

7. Vertical equilibrium

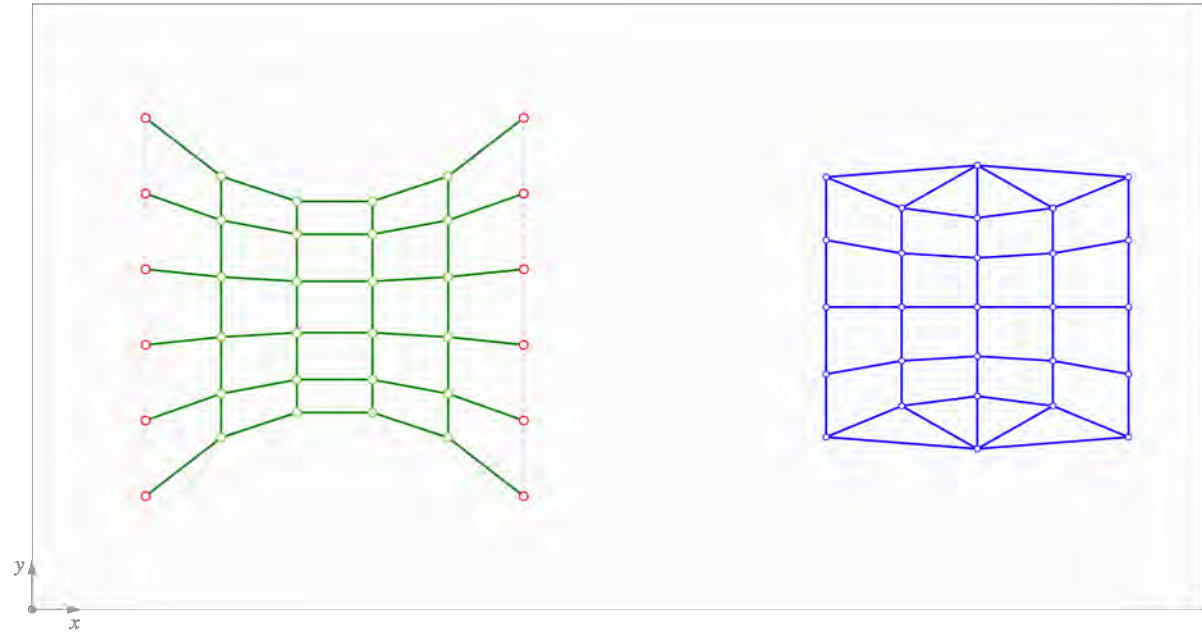
8. Thrust diagram



horizontal equilibrium



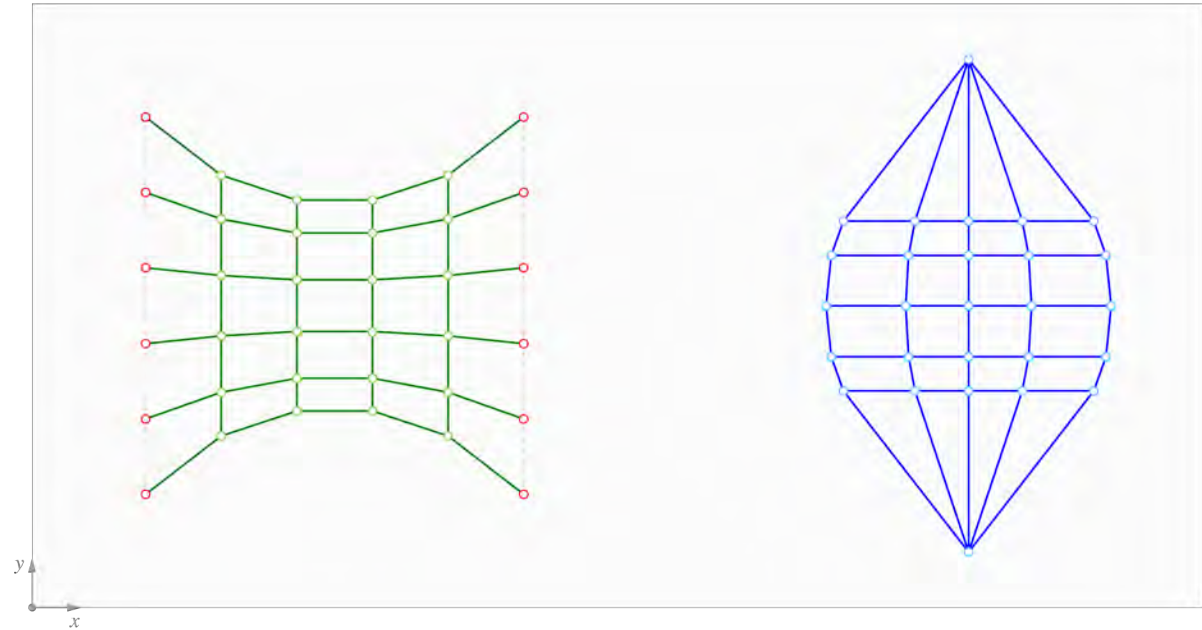
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



horizontal equilibrium



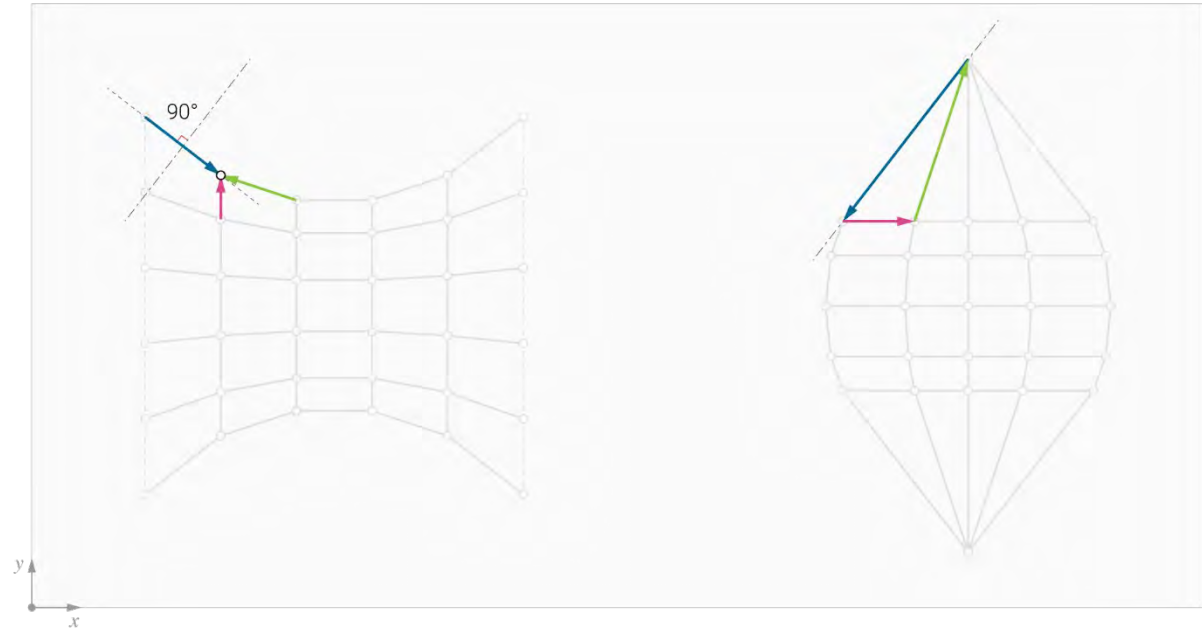
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



horizontal equilibrium



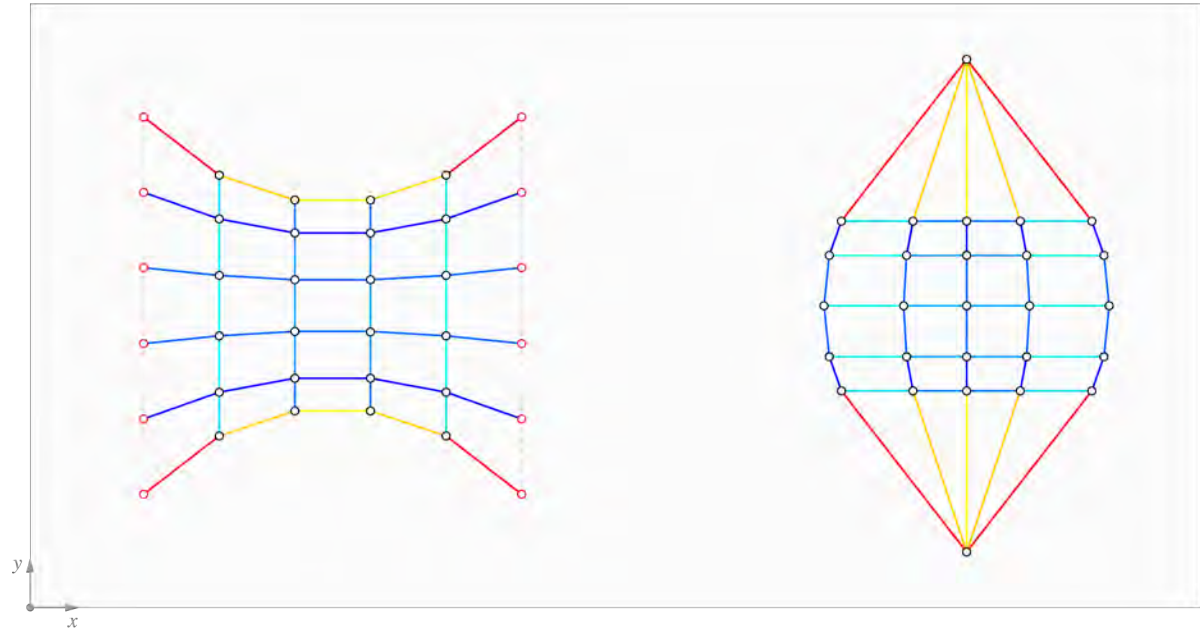
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



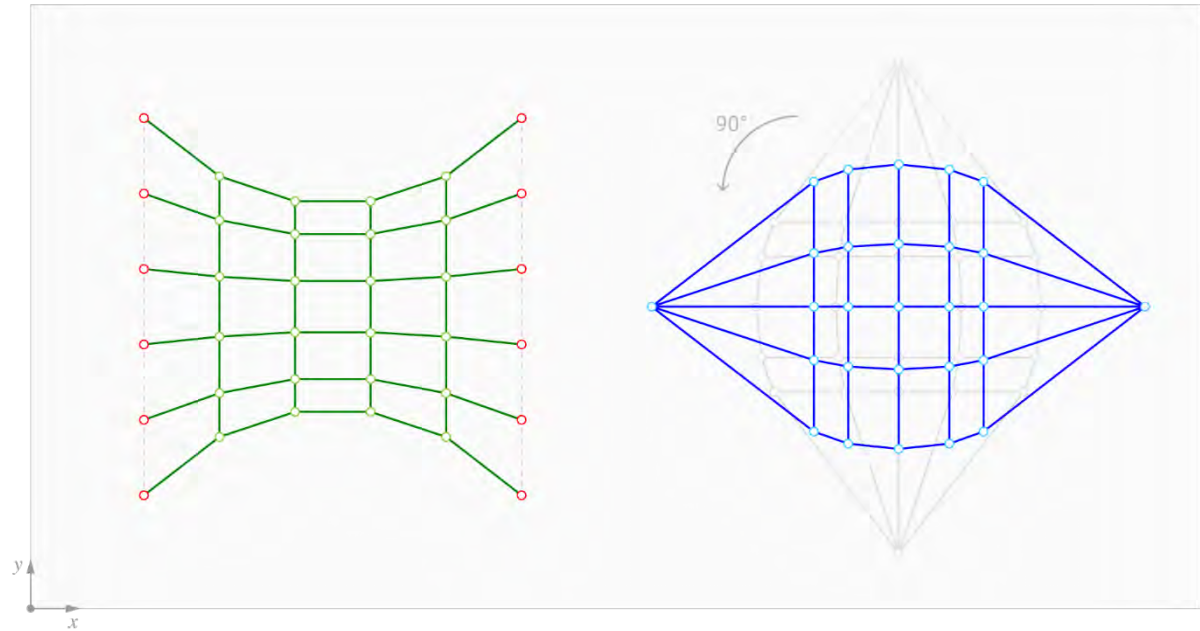
horizontal equilibrium



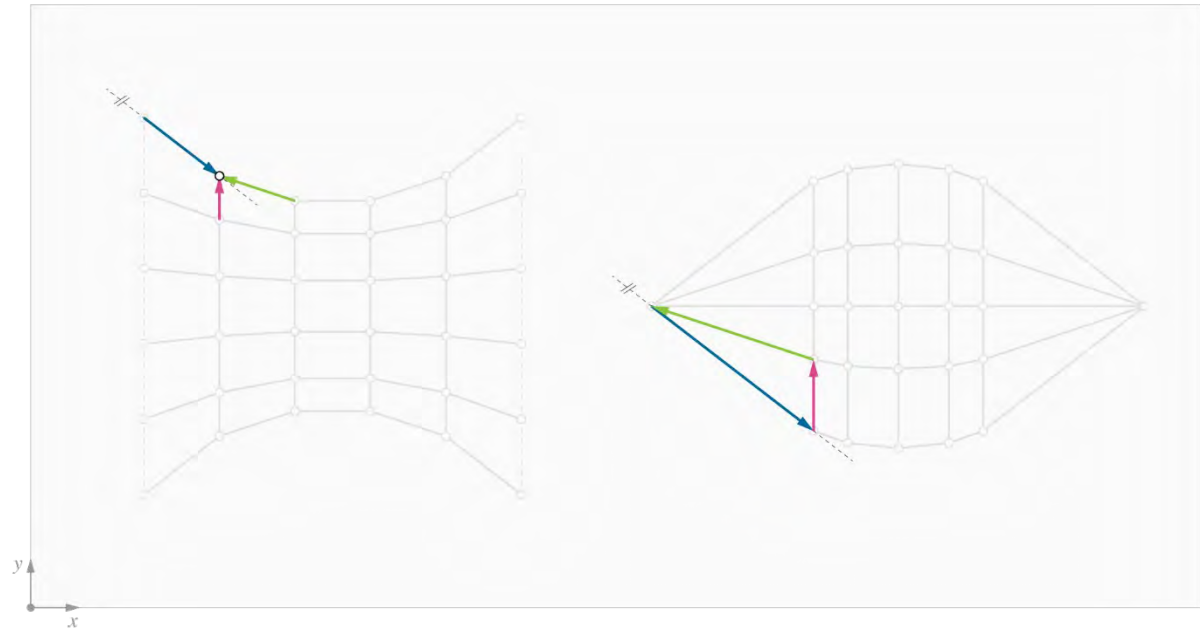
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



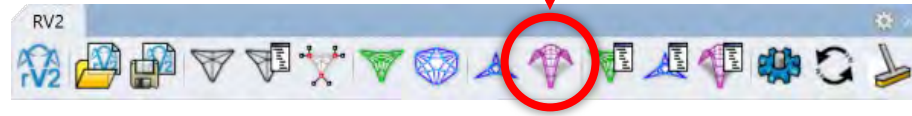
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



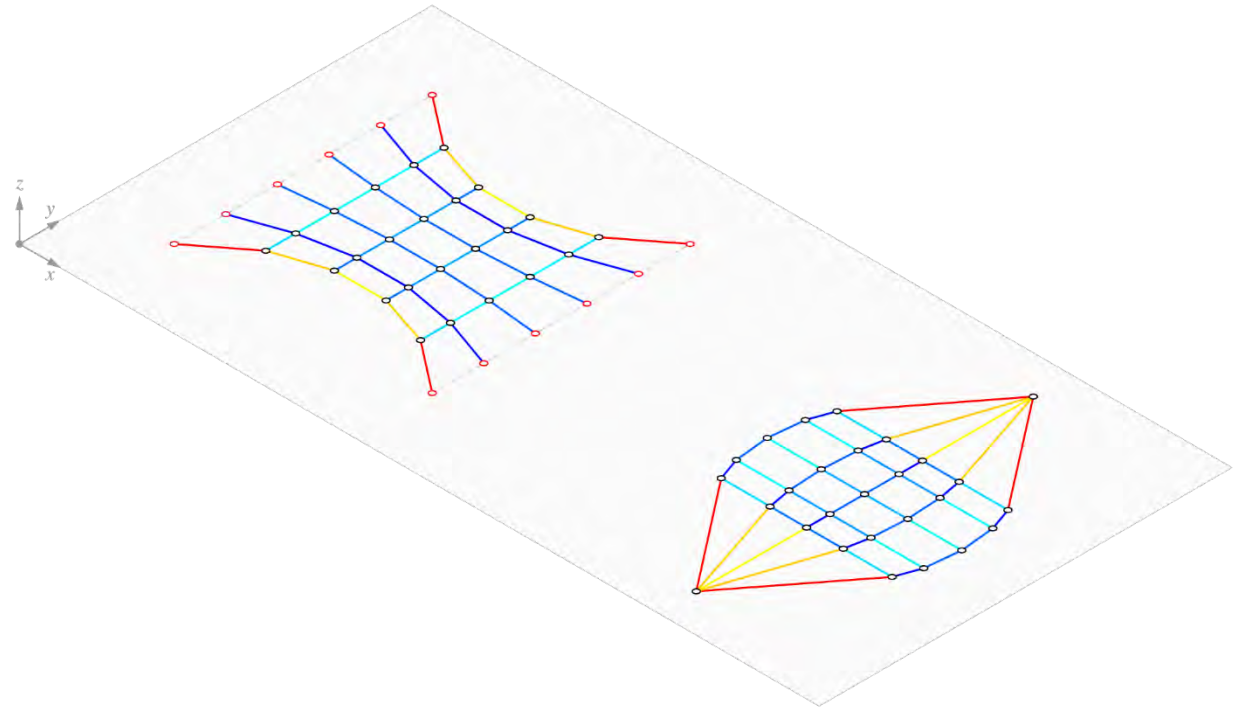
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



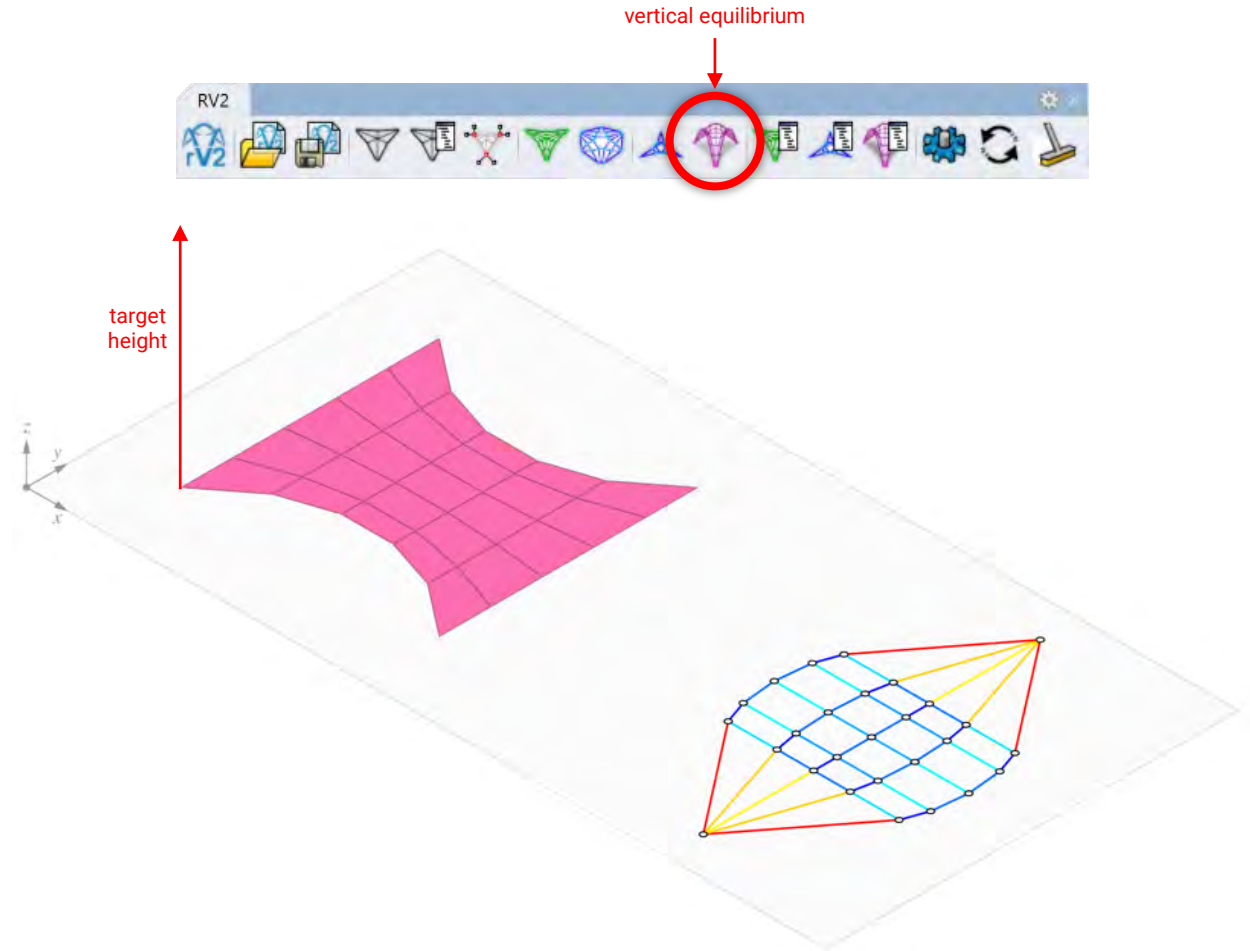
vertical equilibrium



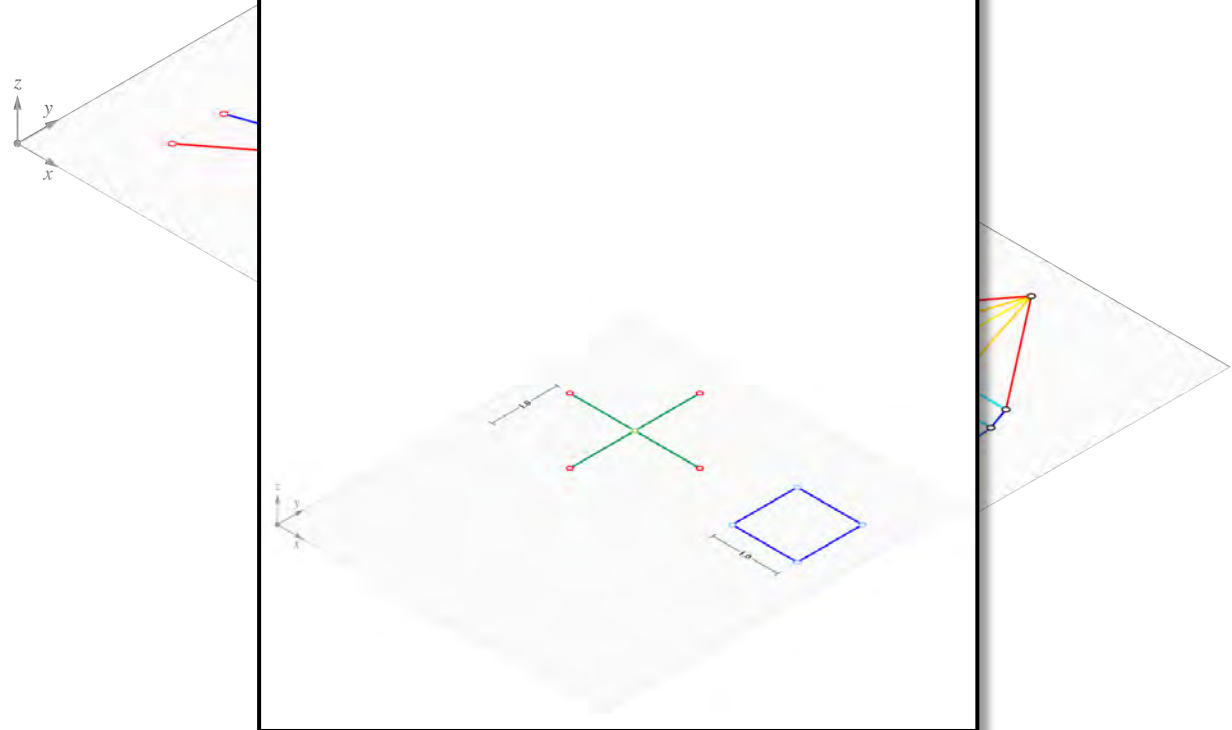
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
- 6. Force diagram**
7. Vertical equilibrium
8. Thrust diagram



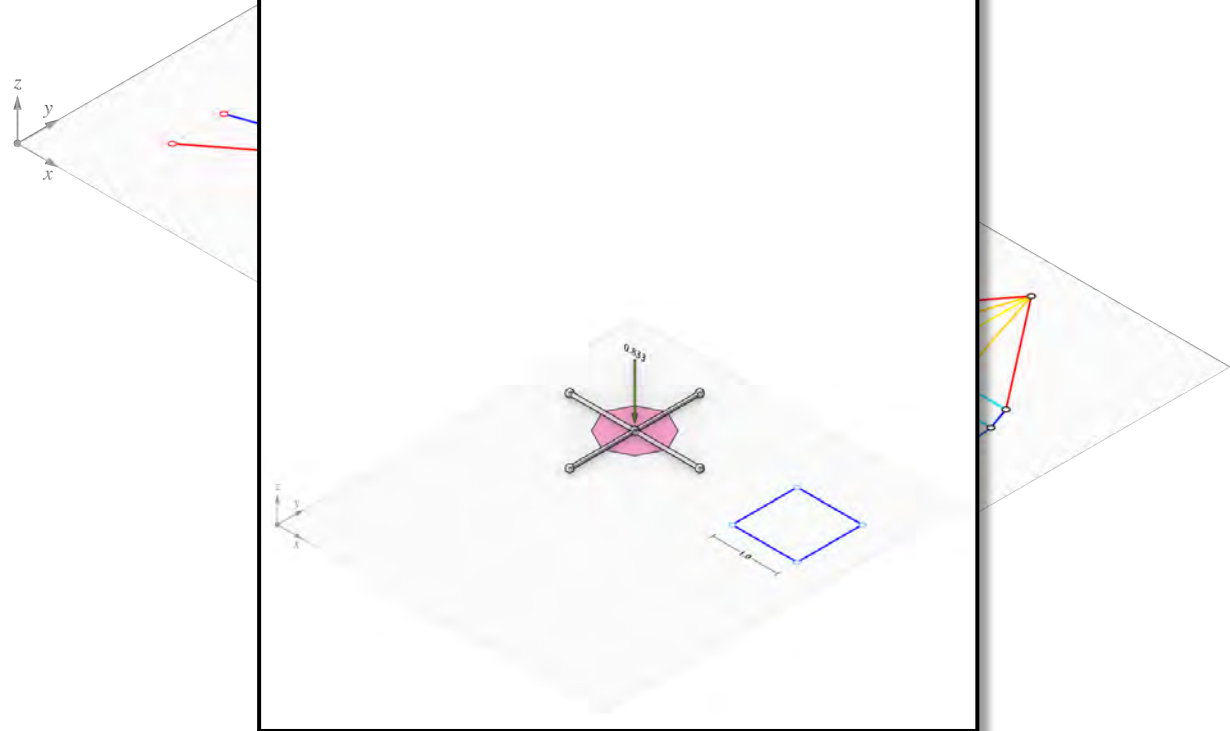
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
- 7. Vertical equilibrium**
8. Thrust diagram



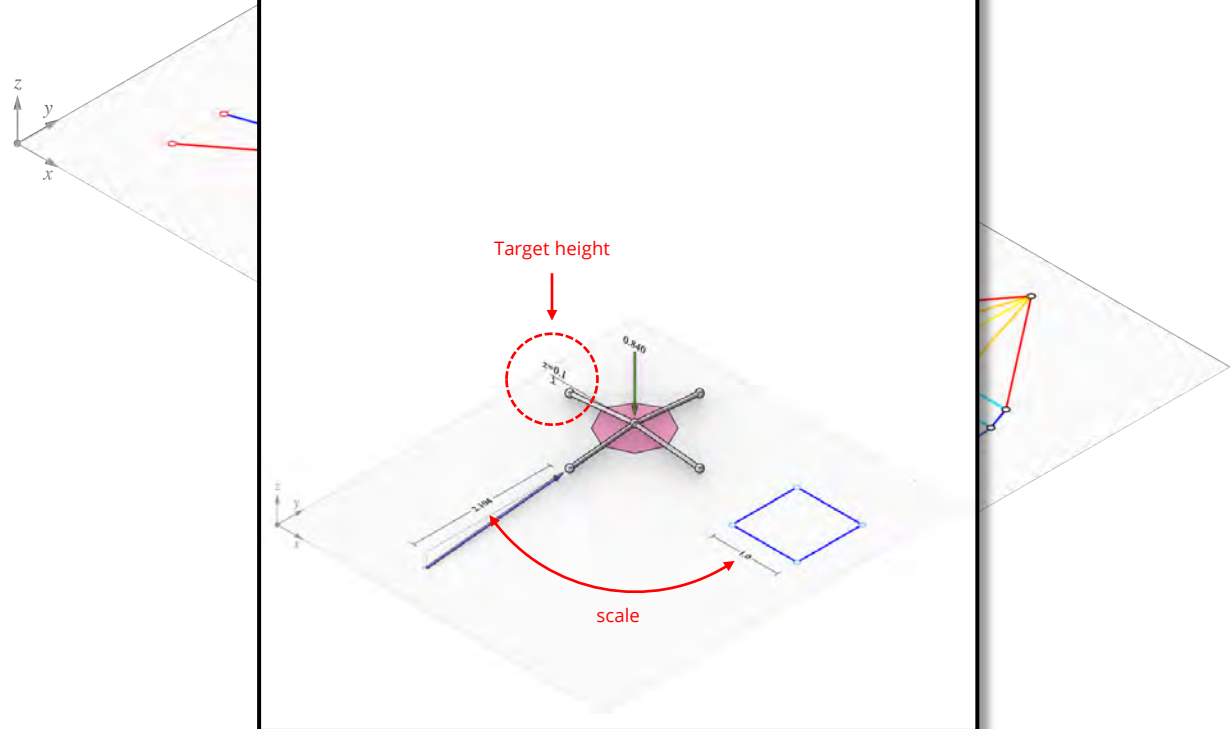
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



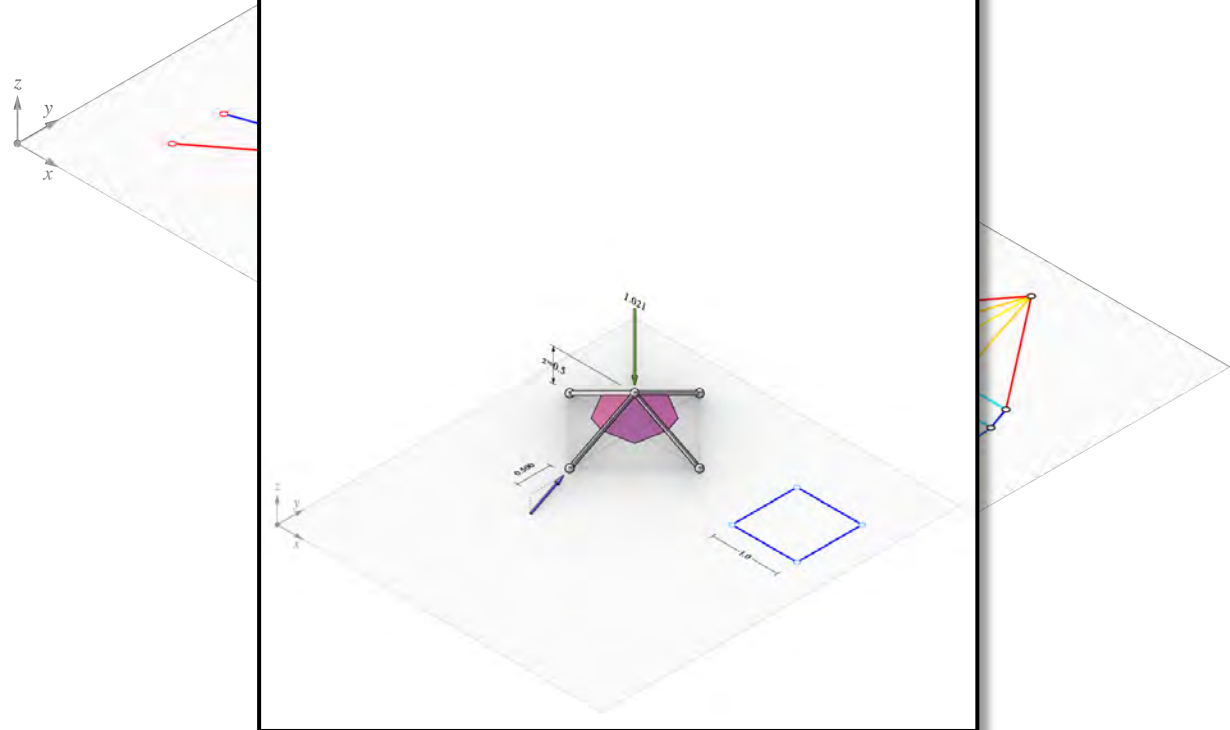
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
- 7. Vertical equilibrium**
8. Thrust diagram



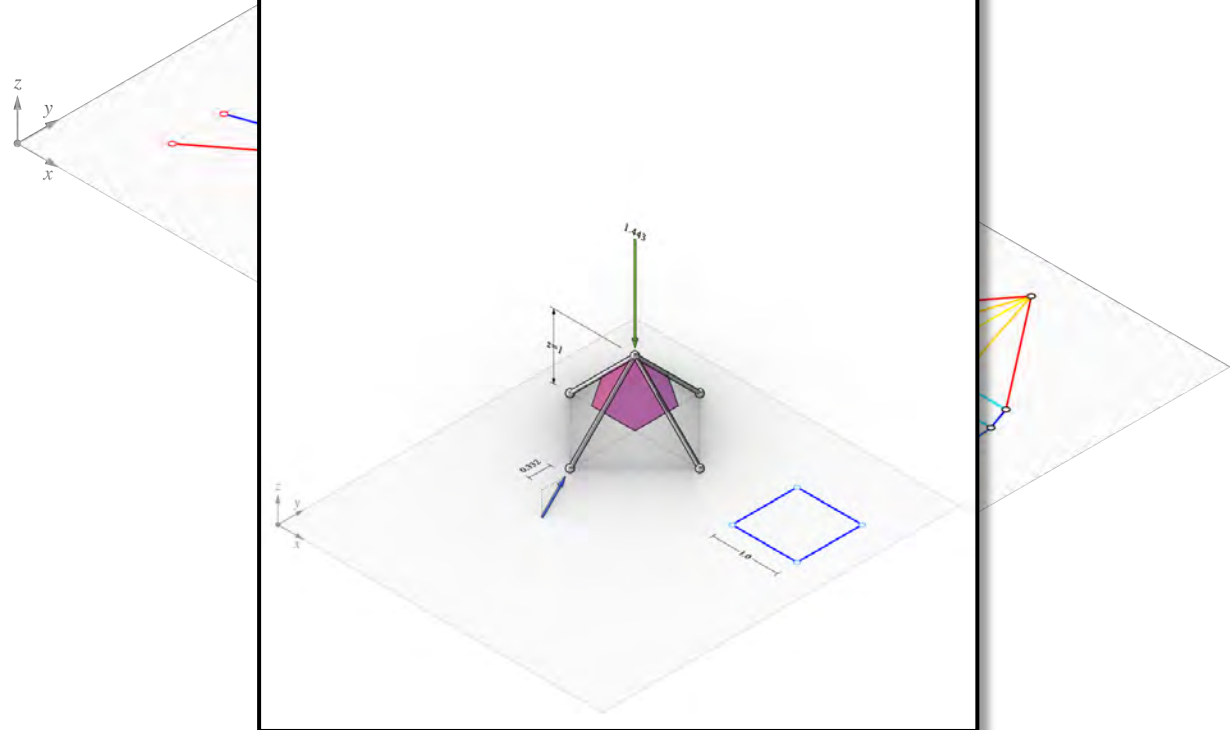
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



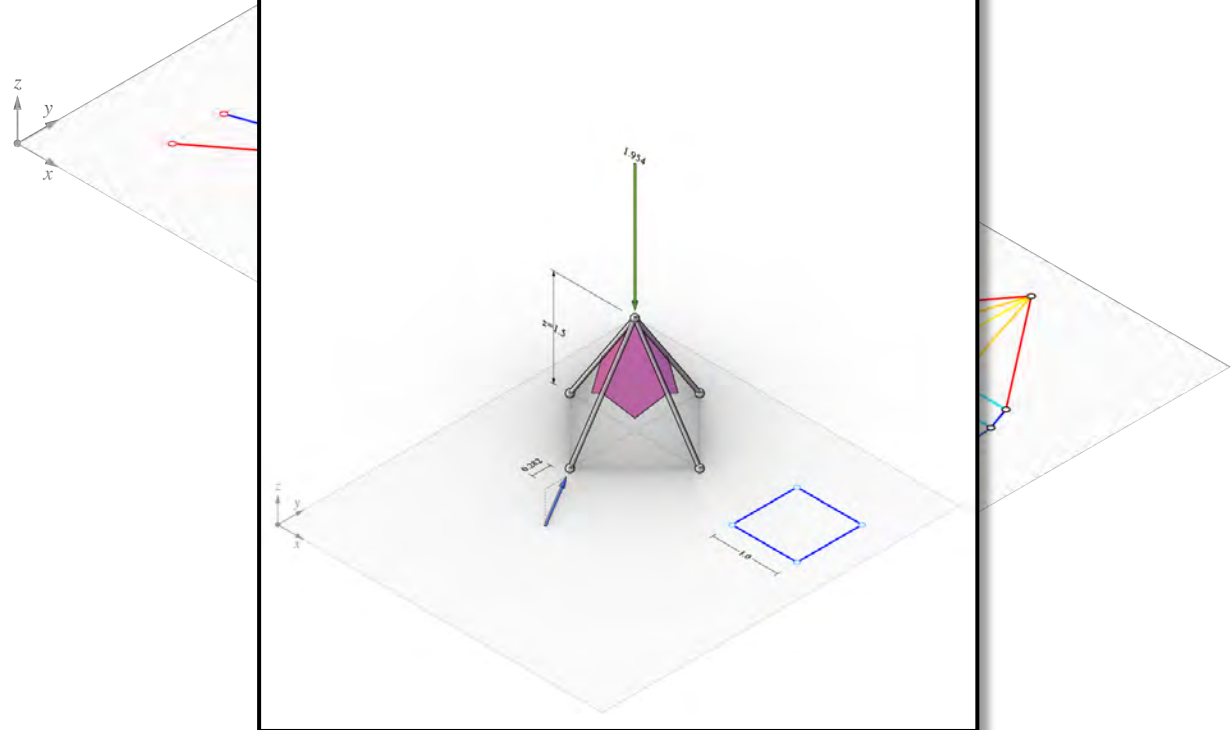
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



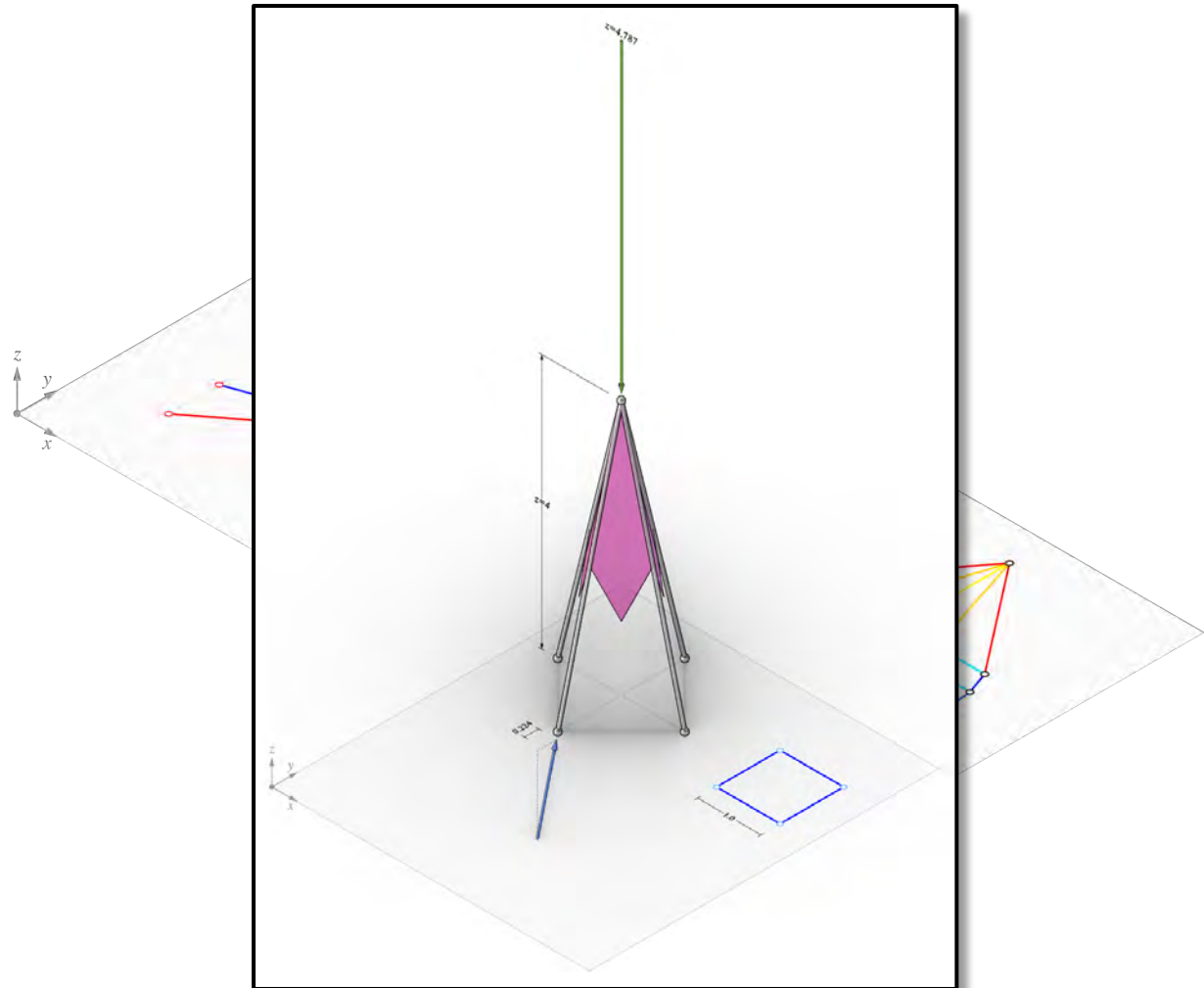
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
- 7. Vertical equilibrium**
8. Thrust diagram



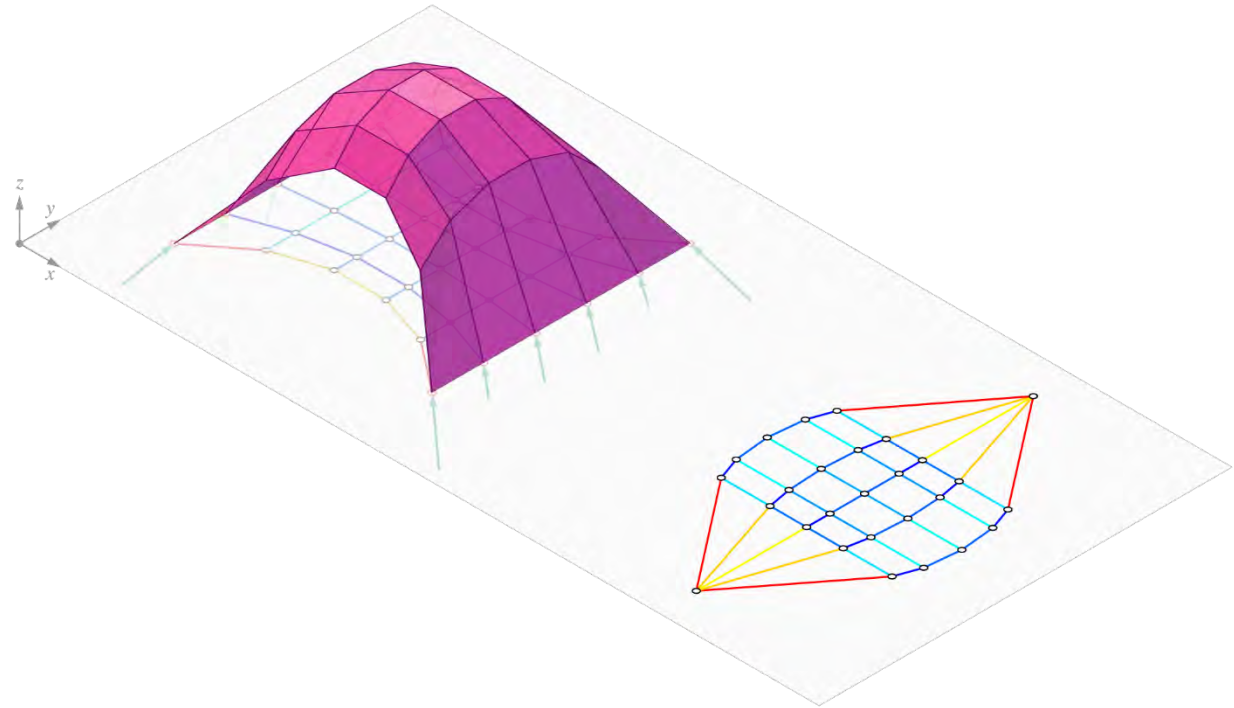
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram



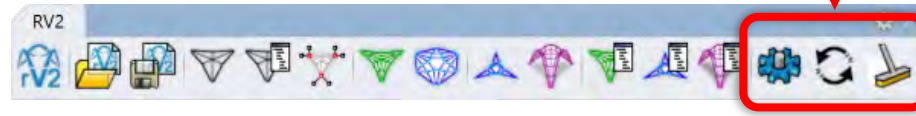
modify diagrams



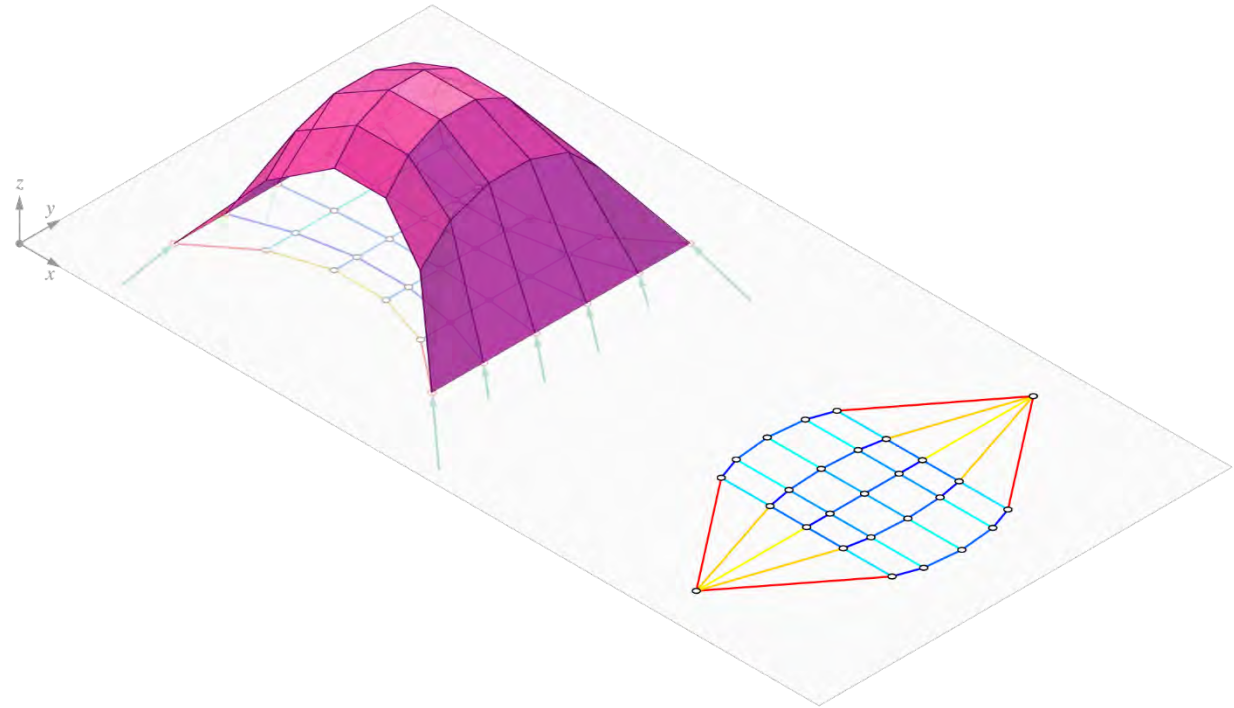
1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram

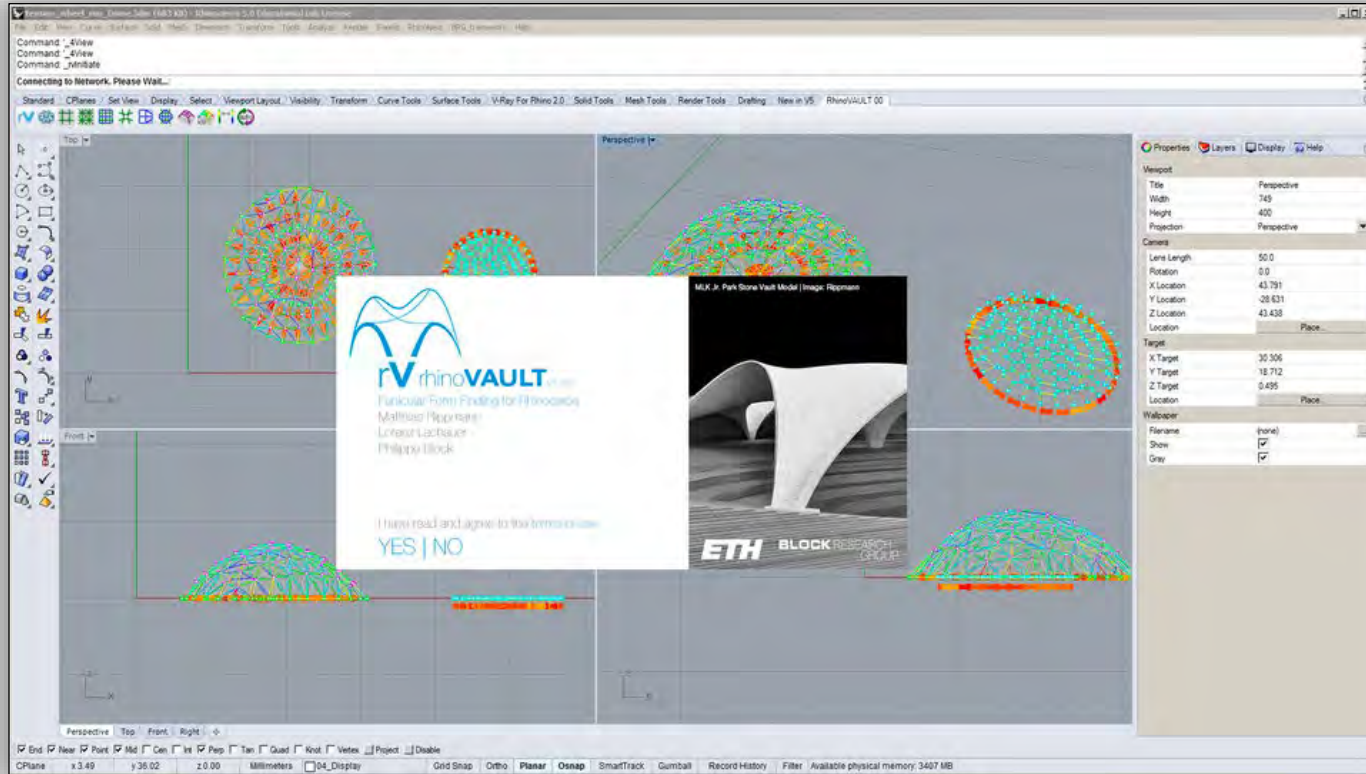


utility functions



1. Pattern
2. Boundary conditions
3. Form diagram
4. Dual diagram
5. Horizontal equilibrium
6. Force diagram
7. Vertical equilibrium
8. Thrust diagram





rhino**VAULT**

MATTHIAS RIPPMANN

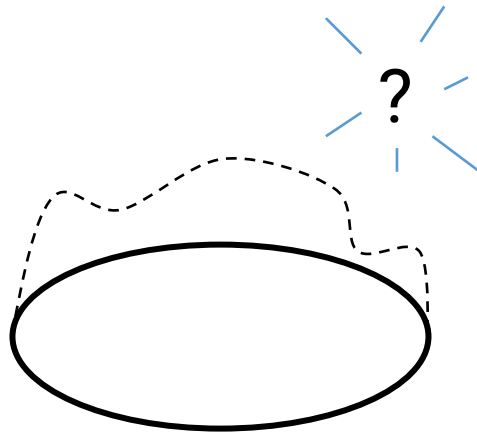
LORENZ LACHAUER

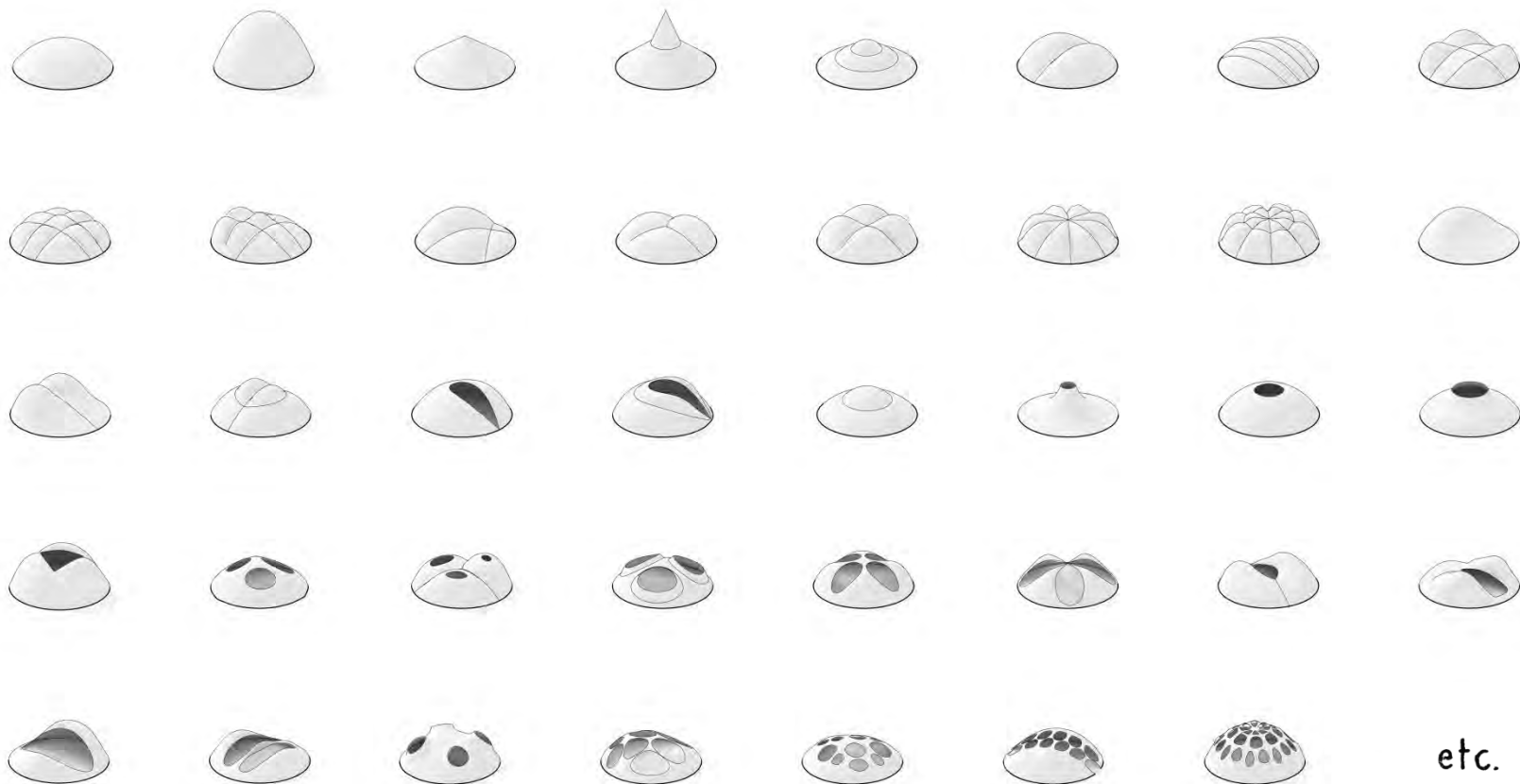
PHILIPPE BLOCK



ETH *Zürich*

BLOCK RESEARCH
GROUP







```
137 if len(nbrs) == 1:
138     u = sorted(nbrs.keys())[0]
139 else:
140     u = sorted(network.halfedge[key].keys())
141 v = _find_first_neighbour(u, network)
142 _find_edge_face(u, v, network)
143 for u, v in network.edges_iter():
144     if network.halfedge[u][0] is None:
145         _find_edge_face(u, v, network)
146     if network.halfedge[v][0] is None:
147         _find_edge_face(v, u, network)
148 _break_faces(network, breakpoints)
149 return network.face
150
151 def _find_first_neighbour(key, network):
152     angles = []
153     nbrs = network.halfedge[key].keys()
154     if len(nbrs) == 1:
155         return nbrs[0]
156     vu = [-1, -1, 0]
157     for nbr in nbrs:
158         w = [network.vertices[nbr][i] for i in 'xyz']
159         v = [network.vertices[key][i] for i in 'xyz']
160         vw = [w[0] - v[0], w[1] - v[1], w[2] - v[2]]
161         angles.append((angle_smallest_vectors(vu, vw)))
162     return nbrs[angles.index(min(angles))]
163
164 def _sort_neighbours(network, ccm=True):
165     sorted_neighbours = {}
166     for key in network.halfedge.keys():
167         nbrs = network.halfedge[key].keys()
168         if len(nbrs) == 1:
169             sorted_neighbours[key] = nbrs[0]
170         else:
171             angles = []
172             vu = [-1, -1, 0]
173             for nbr in nbrs:
174                 w = [network.vertices[nbr][i] for i in 'xyz']
175                 v = [network.vertices[key][i] for i in 'xyz']
176                 vw = [w[0] - v[0], w[1] - v[1], w[2] - v[2]]
177                 angles.append((angle_smallest_vectors(vu, vw)))
178             sorted_neighbours[key] = nbrs[angles.index(min(angles))]
```

```
181 if __name__ == "__main__":
182     import math
183     import random
184
185     from compas.geometry import distance_point_point
186     from compas_rhino.helpers.mesh import draw_mesh
187     from compas.datastructures.mesh import Mesh
188
189     radius = 5
190     origin = (0., 0., 0.)
191     count = 0
192     points = []
193     while count < 1000:
194         x = (random.random() - 0.5) * radius * 2
195         y = (random.random() - 0.5) * radius * 2
196         z = (random.random() - 0.5) * radius * 2
197         pt = x, y, z
198         if distance_point_point(origin, pt) > radius:
199             points.append(pt)
200             count += 1
201     faces = convex_hull(points)
```

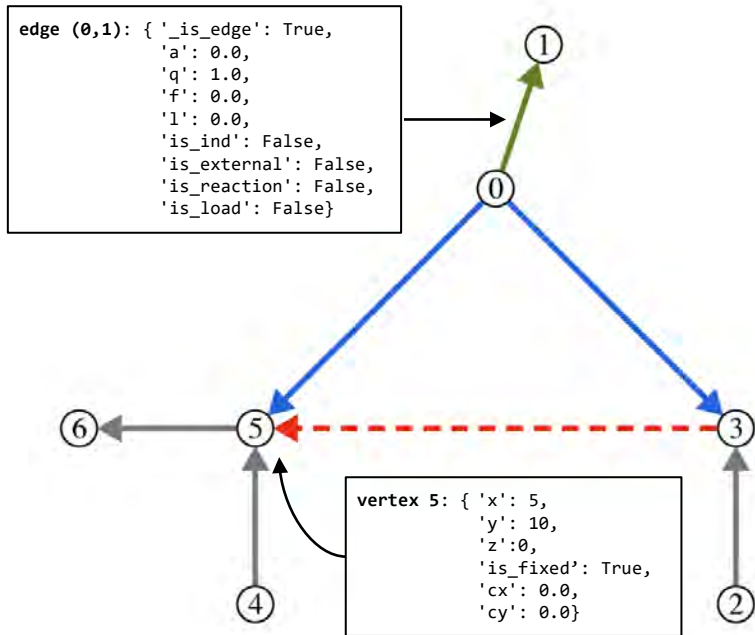


C O M P A S

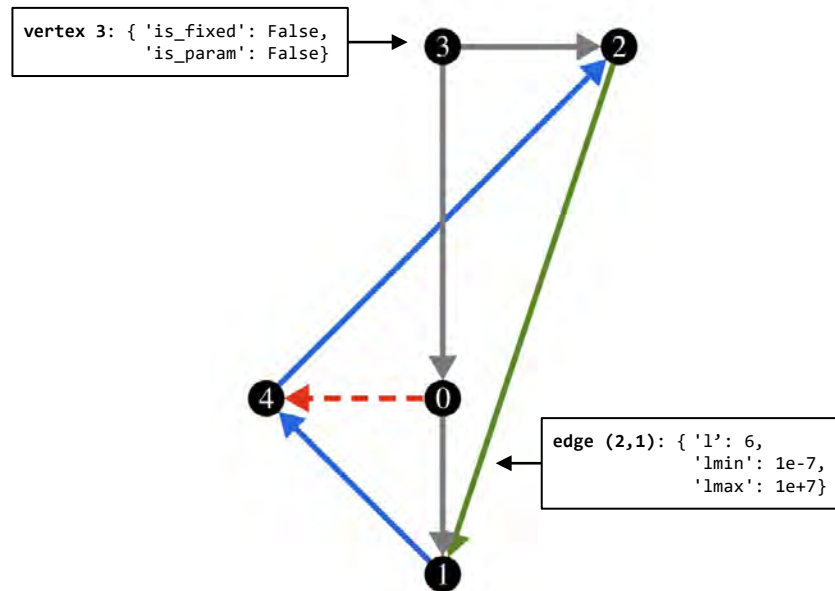
Open-source, Python-based framework for computational research and collaboration
in architecture, engineering and digital fabrication

```
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

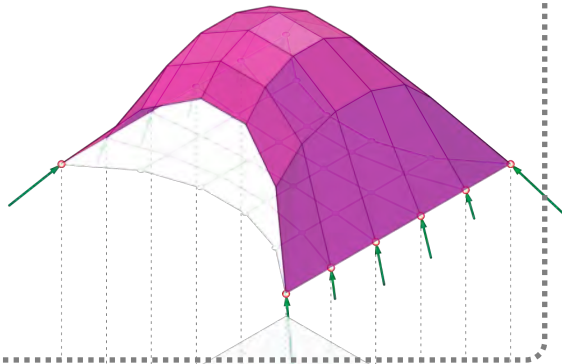
form diagram data



force diagram data



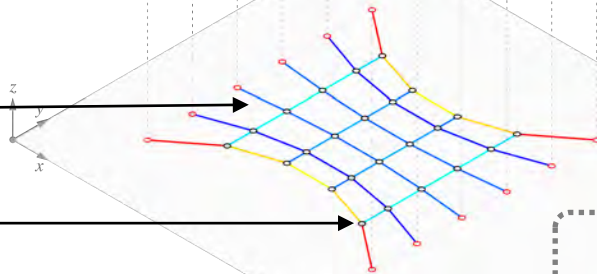
thrust diagram data



form diagram data

```
edge (u, v): {'q': 1.0,  
             'lmin': 0.0,  
             'lmax': 1e+7,  
             ...}
```

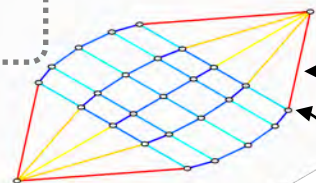
```
vertex: {'is_fixed': False,  
        'is_anchor': False,  
        'x': 0.0,  
        'y': 0.0,  
        'z': 0.0,  
        ...}
```

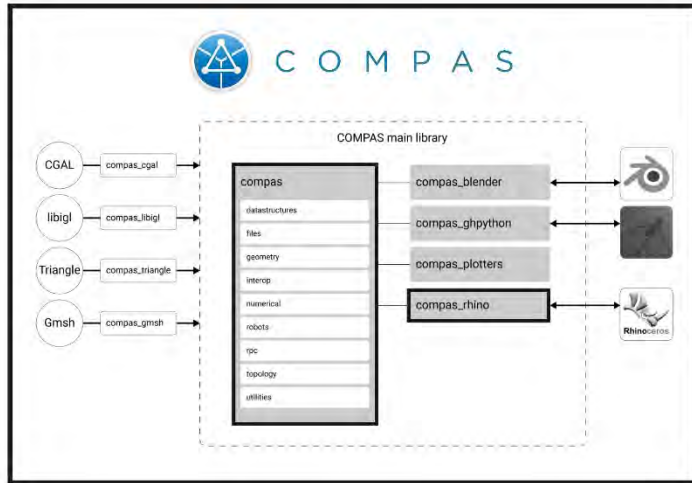


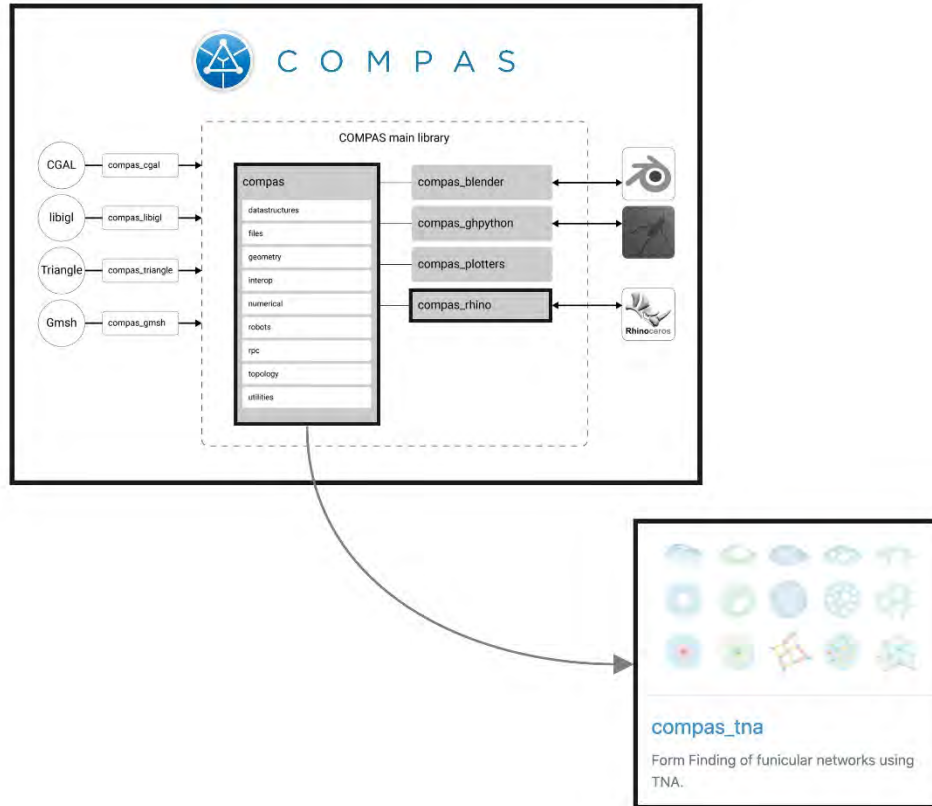
force diagram data

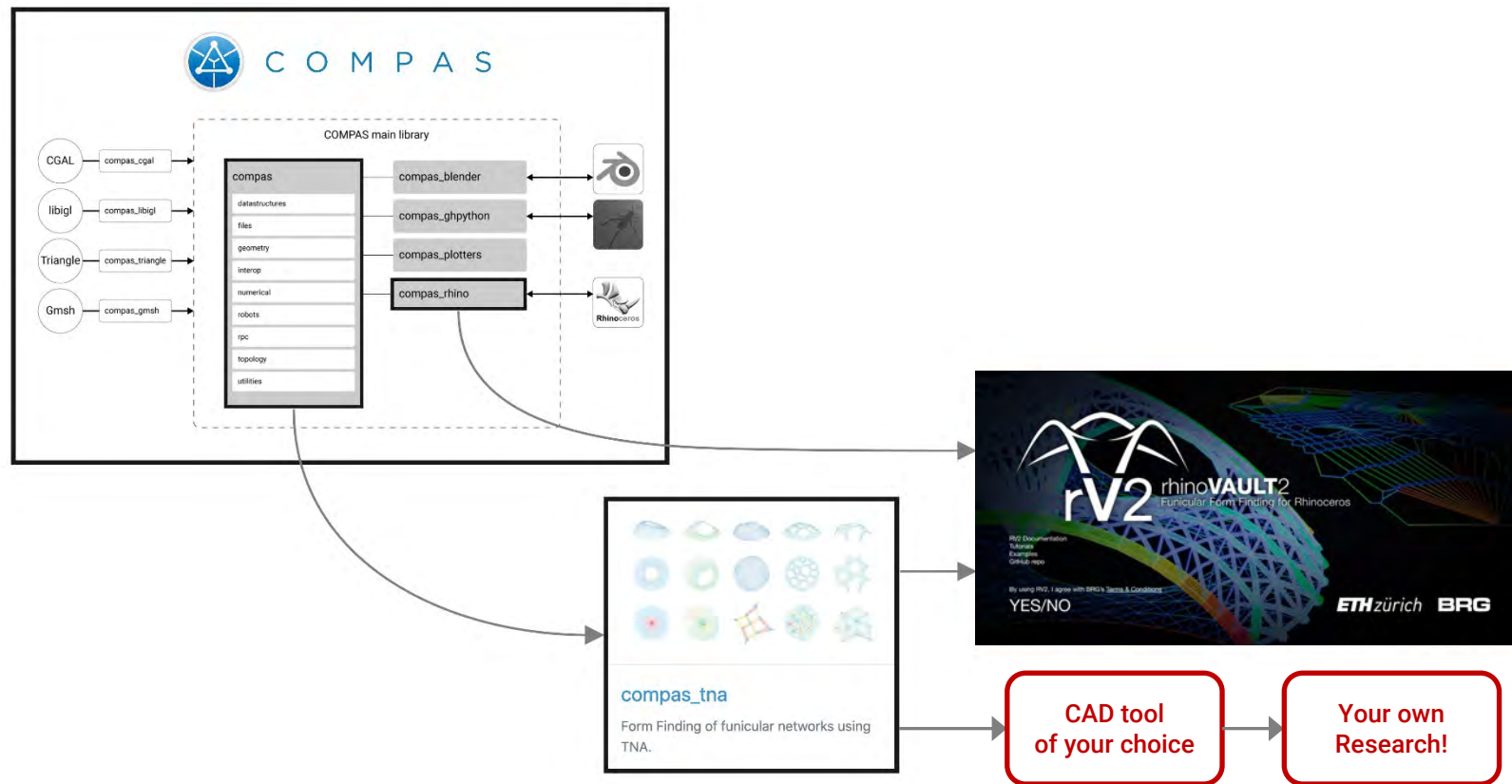
```
edge (u, v): {'a': 0.0,  
             'lmin': 0.0,  
             'lmax': 1e+7}
```

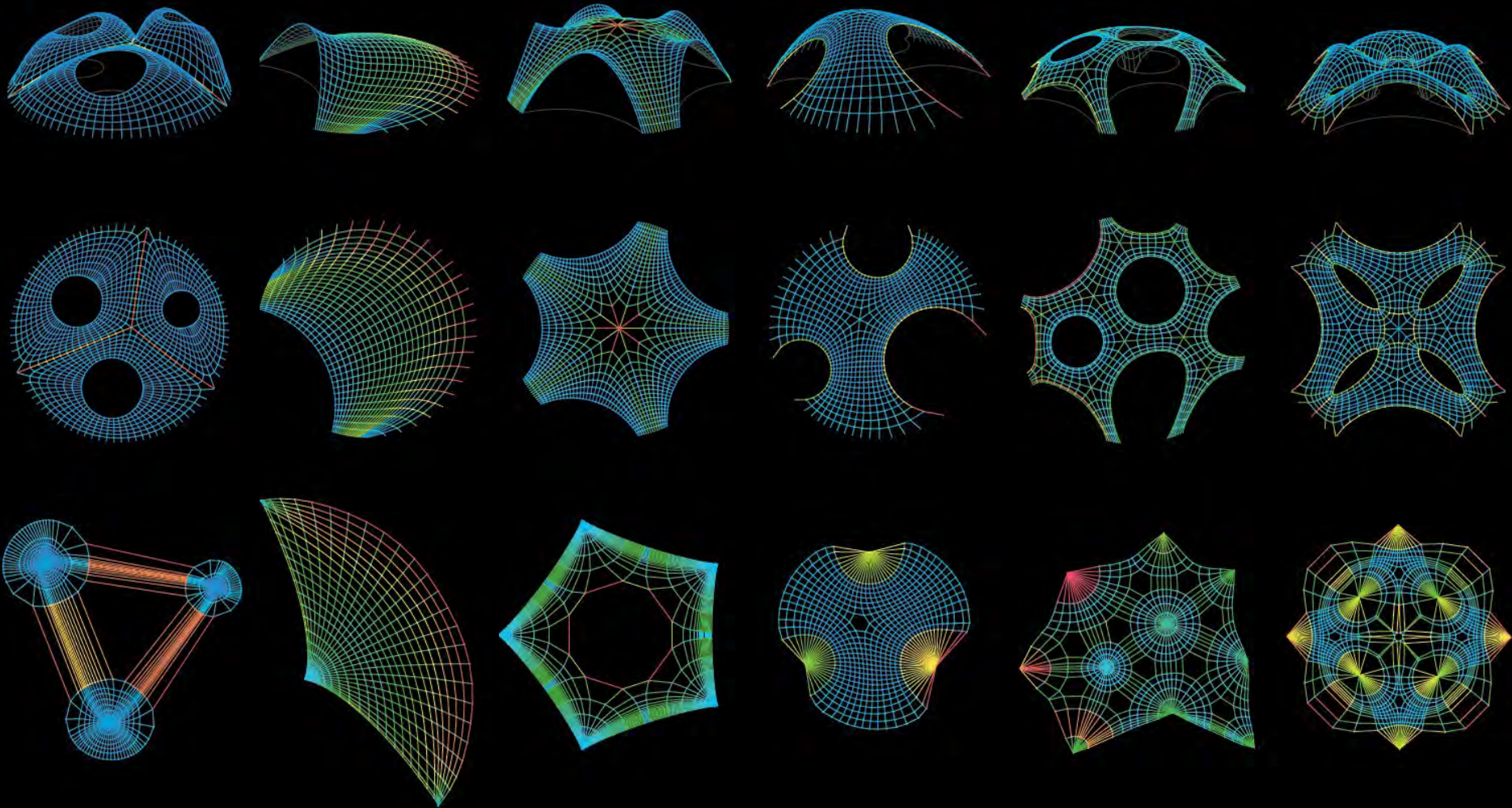
```
vertex: {'is_fixed': False,  
        'x': 0.0,  
        'y': 0.0,  
        'z': 0.0}
```











063-0605-00L : Computational Structural Design 1
Computational Graphic Statics

<http://block.arch.ethz.ch>

 @blockresearchgroup

ETH zürich

DARCH

ETH ZÜRICH
FACULTY OF ARCHITECTURE
DESIGN RESEARCH GROUP

BRG



compass dashboard

v0.7.1

+



RhinoVAULT 2

V1.4.3